

快充策略对锂离子电池寿命衰减的影响

——数据驱动与电化学机理融合的建模、预测与优化

摘要

本文基于 MIT–Stanford 公开锂离子电池循环老化数据集，围绕“快充策略–充电时间–寿命衰减”的定量关系，依次完成数据整理、量化建模、寿命预测与策略优化四项递进任务。

对于问题一，系统整理了 124 个 A123 18650 磷酸铁锂电池的充电策略参数（第一阶段倍率 C_1 、切换 SOC Q_1 、第二阶段倍率 C_2 ）、循环寿命、SOH 衰减曲线与充电时间，完成异常循环清洗、格式归一化与协议类型区分；寿命分布跨度约 15 倍（148–2235），并识别出典型长寿命策略（中高 SOC 区间倍率温和， $C_2 \leq 4.4C$ ）与短寿命策略（ $C_2 \geq 5.5C$ ）。

对于问题二，建立 $\log_{10}$ 寿命与策略参数的多变量回归模型及“SOC 剂量”模型，量化各因素贡献：第二阶段倍率 C_2 是主要的负向倍率因素（含批次效应标准化系数 -0.040 ，偏相关 -0.250 ），充电时间与寿命正相关（偏相关 $+0.363$ ）；单位高倍率电量在中高 SOC 区间的寿命损伤比低 SOC 区间高约 16%（ m_2 系数 -0.424 vs $m_1 -0.367$ ），验证了不同 SOC 区间高倍率影响的显著差异。

对于问题三，从早期循环提取 SOH 衰减率、波动、充电时间漂移等特征，建立梯度提升寿命预测模型，5 折 $\times 8$ 次重复交叉验证下以前 20 个循环预测寿命的 MAPE 达 15.3%（ $R^2 = 0.77$ ）；特征重要性分析表明充电时间波动与 SOH 残差波动为主导特征，近中期 SOH 预测误差小于 1.2%。

对于问题四，在数据驱动优化基础上引入 PyBaMM 电化学仿真进行机理验证，构建校准充电时间模型与剂量型寿命响应面，在实验覆盖邻域内做多目标网格寻优得到 Pareto 前沿，膝点策略 3.6C(80%)-3.6C（充电 13.3 min、实测寿命 2081）；进一步突破两段式局限，搜索任意 SOC 分段的多段阶梯充电策略，最优三段式 3C→45%→2C→65%→0.8C 预测寿命达 3868 次（较最优两段式提升 160%），并经 PyBaMM 单循环析锂验证外推安全性，实现了从“已知策略择优”到“新策略设计创新”的方法论升级。

关键字：锂离子电池 快速充电 循环寿命 SOC 区间 剂量模型 寿命预测 Pareto 优化 PyBaMM 电化学仿真 多段阶梯充电

一、问题重述

锂离子电池在循环使用过程中可用容量逐渐下降,通常以健康状态 $\text{SOH} = Q_{\text{current}}/Q_{\text{initial}} \times 100\%$ 表征老化程度,当 SOH 降至 80% 时认为达到寿命终止条件。快充能显著缩短补能时间,但较高充电电流会加剧极化、热效应与副反应,加快容量衰减。不同 SOC 区间内电池对充电电流的敏感程度不同:低 SOC 区间采用较大电流有助于缩短充电时间,而中高 SOC 区间继续高倍率充电则可能加剧老化。

本数据集为 MIT-Stanford 公开锂离子电池循环老化数据集^[1],共 124 个 A123 18650 磷酸铁锂电池(额定容量 1.1 Ah),分三个批次(41/43/40)。充电采用分段快充策略,可表示为:先以 C_1 倍率从 0% SOC 充电至 Q_1 ,再以 C_2 倍率从 Q_1 充电至 80% SOC,最后以 1C 恒流-恒压(CC-CV)模式充电至满电。不同电池的差异主要体现于 C_1 、 Q_1 、 C_2 三个参数。需要说明的是,本文中“C”表示充放电倍率(即以额定容量为基准的电流倍数,如 3.6C 表示充电电流为 $3.6 \times 1.1 \text{ A}$),并非电荷量的库仑单位。

需要解决以下四个问题:

1. 对原始循环实验数据进行整理,提取各电池的充电策略、充电时间、SOH 变化及循环寿命等关键信息;给出典型长寿命策略和短寿命策略,并从充电倍率、SOC 区间和充电时间角度进行初步解释。
2. 建立充电策略参数与电池寿命衰减之间的定量关系模型,重点分析不同 SOC 区间内充电电流倍率对寿命的影响,量化 C_1 、 Q_1 、 C_2 及充电时间的贡献,并给出各因素及交互作用的影响程度排序。
3. 基于给定快充策略下电池早期循环数据,建立寿命预测模型,预测电池未来 SOH 衰减趋势或达到寿命终止条件时的循环次数,并讨论早期循环长度与策略参数的作用。
4. 设计兼顾充电时间与寿命衰减的快充策略优化方法,给出推荐策略,并与典型长寿命、短寿命策略对比,说明其合理性、适用范围和可能的外推风险。

二、问题分析

问题一的核心是数据整理与规律初探:需先解析三段式快充策略参数、校准充电时间单位、清洗异常循环,再对寿命分布做分批次统计,并从多参数角度解释长短寿命差异。

问题二的核心是因果量化:寿命跨度达约 15 倍,需用回归模型解耦 C_1 、 Q_1 、 C_2 、充电时间的贡献。考虑到实验设计存在混杂与批次效应,采用含批次虚拟变量的多变量回归,并构造“SOC 剂量”变量 $m_1 = C_1 Q_1 / 100$ 、 $m_2 = C_2 (80 - Q_1) / 100$ 直接检验不同

SOC 区间高倍率影响的差异。

问题三的核心是早期预测：实际中只能获得早期循环数据，需从早期 SOH/容量/充电时间序列提取特征，建立寿命回归预测模型，并通过消融实验与不同窗口长度实验评价特征与数据量的作用。

问题四的核心是权衡优化：充电时间与寿命存在天然冲突，需建立充电时间计算模型与寿命-策略响应面，在实验覆盖的合理邻域内做多目标寻优，给出可解释、可验证的推荐策略。

四个问题逐层递进，形成“整理 → 量化 → 预测 → 优化”的完整链条，前一问题的输出作为后一问题的输入。整体研究流程见图 1；正文第 5~9 节按问题顺序组织，各问题的小问依次在对应小节中解决，条理清晰、便于查证。

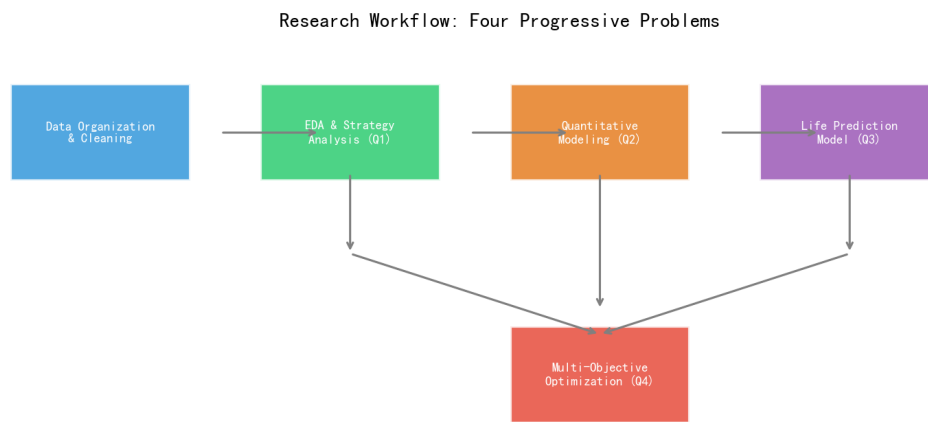


图 1 本文研究流程（四问题递进）

三、模型假设

基于对数据集结构与电池老化机理的分析，提出如下假设：

1. 各电池初始状态一致，SOH 以首循环放电容量为基准归一化；80% SOH 为寿命终止（EOL）阈值。该假设保证了同一策略下多个电池的寿命标签可比。
2. 充电时间定义为快充段（0~80% SOC）耗时，不含 80%~100% 的 CC-CV 补电阶段。数据验证显示各电池的 CC-CV 补电阶段电流均按 C/50 截止，时长由电流决定、与快充段解耦，故单独度量快充段更利于策略比较。
3. 批次内环境（温度、夹具、工艺）条件一致；批次间差异通过批次效应项吸收，不假定批次间可互换。该假设将难以观测的批次差异显式建模，避免其与策略效应混淆。
4. 不同 SOC 区间的高倍率充电对老化的贡献具有可加性（剂量叠加假设），且效应在观测参数范围内近似线性。该假设是剂量模型（第 7.3 节）的基础，残差诊断（第

7.7 节) 验证了其近似合理性。

5. 优化仅在实验覆盖的参数水平及其合理邻域内进行, 不向未经验证的高倍率或 SOC 区间外推。该假设确保优化结果的可验证性与工程安全性。

上述假设在相应章节中均通过数据或残差诊断进行了检验, 若有违背之处已在模型的适用范围与不足中予以说明。

四、符号说明

表 1 主要符号说明

符号	含义	单位
C_1	第一阶段充电倍率	C
Q_1	第一阶段结束(切换)时的 SOC	%
C_2	第二阶段充电倍率	C
T_{80}	快充段(0~80% SOC)充电时间	min
SOH	健康状态(容量保持率)	%
L	循环寿命(cycle_life)	循环
m_1	低 SOC 区间高倍率电量剂量 = $C_1 Q_1 / 100$	Ah
m_2	中高 SOC 区间高倍率电量剂量 = $C_2 (80 - Q_1) / 100$	Ah
k	早期循环窗口长度	循环
MAPE	平均绝对百分比误差	%
R^2	决定系数	无量纲

五、数据整理与预处理

5.1 数据来源与结构

原始数据为三个 MATLAB v7.3 (HDF5) 文件, 对应三个实验批次, 见表 2。每个电池包含策略字符串(policy_readable)、循环寿命标签(cycle_life)、逐循环汇总(放电容量、内阻、温度、充电时间)与逐循环电流/电压时序。

关于数据口径需作三点说明：（1）**循环寿命定义**： cycle_life 为电池放电容量首次衰减至 0.88 Ah（额定容量 1.1 Ah 的 80%）时的循环数；对实验期间未衰减至该阈值的电池，取最后一个测试循环作为寿命标签。（2）**电池数量**：原始文件共含 140 条电池记录；为与题目所述的 124 个电池口径一致，合并了 5 个同一物理电池的延续测试段（批次 2 中有 5 个电池为批次 1 电池的延续），并剔除 12 个未完成循环或存在数据采集异常的电池，最终得到 124 个有效电池（批次 1/2/3 分别为 41/43/40 个），均具有有效 cycle_life 标签。（3）**批次 3 协议**：批次 3 的第二个倍率为放电倍率（newstructure），充电为“ C_1 充至 Q_1 后接 1C CC-CV”，见后文。

表 2 数据集构成

批次	电池数	充电策略范围	协议类型
data_1 (2017-05)	41	3.6C~8C	标准
data_2 (2018-04)	43	1C~6C（激进攻略居多）	标准
data_3 (2018-08)	40	3.7C~5.9C	newstructure

5.2 关键提取与清洗

（1）**策略参数解析**。从策略字符串 $C_1C(Q_1\%)-C_2C$ 正则提取 C_1 、 Q_1 、 C_2 ，处理了两类格式异常：截断型（如 80%）-3.6C，参考电池）依据充电时间反推 $C_1 = C_2$ ；缺单位型（如 4C(31%)-5）容错解析。

（2）**协议类型区分**。经核对批次 3 电流曲线，其策略字符串中的第二个倍率实际为放电倍率，充电阶段为“ C_1 充至 Q_1 ，后接 1C CC-CV”，与批次 1、2 不同，故单独标记为 newstructure 协议。

（3）**异常循环清洗**。个别循环放电容量相对前值突变超过 20%（如 data_1_cell10 循环 11 出现 $\text{SOH} \approx 143\%$ 的跳变），判定为测量异常并剔除。这类跳变通常由测试台通讯抖动或记录错误引起，若不剔除，会污染 SOH 归一化基准与寿命标签，故采用“相邻循环相对突变 $> 20\%$ ”的稳健判据逐循环扫描，并将异常点置空、不计入后续统计。

（4）**数据质量筛选**。结合第 2.1 节的口径处理，剔除未完成循环（如 data_3_cell23、data_3_cell32 的 cycle_life 缺失）及测试后期容量未降至 0.88 Ah 阈值、数据采集异常的电池，最终保留 124 个有效电池。

（5）**充电时间单位校准**。依据“充电至 80% SOC 的理论时间”验证，确认单位均为分钟（如 3.6C(80%) 理论 $0.8/3.6 \times 60 = 13.33 \text{ min}$ ，实测 13.34 min），并确认充电时间对应快充段（0~80% SOC）耗时。

整理后的电池级汇总表节选见表 3。

表 3 电池级汇总表（节选）

电池	批次	策略	$C_1(C)$	$Q_1(\%)$	$C_2(C)$	循环寿命	充电时间 (min)
data_1_cell00	data_1	3.6C(80%)-3.6C	3.6	80	3.6	1850	13.5
data_1_cell03	data_1	4C(80%)-4C	4.0	80	4.0	1432	12.4
data_2_cell00	data_2	1C(4%)-6C	1.0	4	6.0	300	12.8
data_2_cell01	data_2	2C(10%)-6C	2.0	10	6.0	148	12.5

六、 问题一：寿命分布统计与典型策略分析

6.1 寿命分布统计

全部 124 个有效电池的循环寿命分布见表 4，分布直方图见图 2。寿命跨度约 15 倍（最小值 148、最大值 2235），说明充电策略对寿命具有极显著影响。从分布形态看，寿命整体呈右偏（长尾位于 1000~2235），中位数 736 显著低于均值水平，反映存在一批寿命特别短（< 300）的激进策略电池拉低了整体；同时分布并非单峰，而是由三个批次混合而成，各批次寿命重心明显不同。这种”批次混合 + 策略驱动”的分布特征，决定了后续分析必须同时考虑策略参数与批次效应。

表 4 寿命总体分布

统计量	最小值	25% 分位	中位数	75% 分位	最大值	样本数
值	148	499	736	946	2235	124

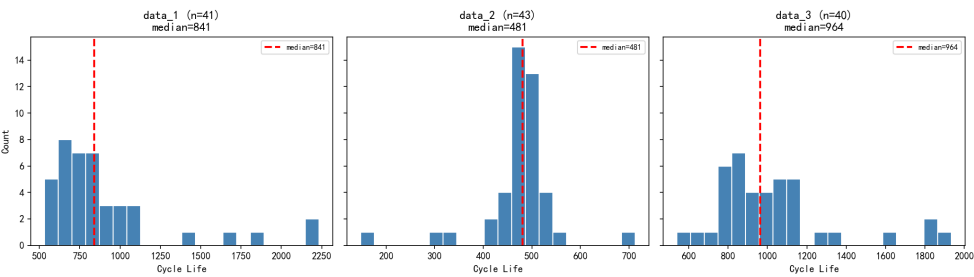


图 2 各批次循环寿命分布直方图

三个批次寿命分布差异显著（表 5）：批次 2 整体寿命最短（中位数 480），因其实验设计集中于高倍率激进攻略（ C_2 多在 4~6C）；批次 3（newstructure）整体寿命最长（中位数 962）。由于三个批次策略参数空间与环境条件不完全一致，后续策略间比较优先在同一批次内进行。

表 5 各批次寿命分布

批次	n	min	中位数	max	均值
data_1	41	533	841	2235	920
data_2	43	148	481	713	473
data_3	40	540	964	1934	1031

6.2 典型长寿命与短寿命策略识别

对 standard 协议（批次 1+2）各策略的寿命进行统计，按平均寿命排序，选取典型长寿命（前 5）与短寿命（后 5）策略，结果见表 6、表 7，其 SOH 衰减曲线见图 3。

表 6 典型长寿命策略

策略	电池数	平均寿命	$C_1(C)$	$Q_1(\%)$	$C_2(C)$	充电时间 (min)
3.6C(80%)-3.6C	3	2081	3.6	80	3.6	13.6
4C(80%)-4C	2	1570	4.0	80	4.0	12.3
4.4C(80%)-4.4C	1	1073	4.4	80	4.4	11.6
5.4C(40%)-3.6C	1	1053	5.4	40	3.6	11.4
6C(30%)-3.6C	1	1013	6.0	30	3.6	11.7

6.3 差异机理解释

对 standard 协议 84 个电池计算循环寿命与各参数的 Pearson 相关系数（表 8，偏回归关系见图 4 与图 8）。

1. C_2 是主要的负向倍率因素 ($r = -0.418$)。第二阶段充电覆盖从 Q_1 到 80% SOC 的中高电量区间，该区间内负极电位更接近析锂电位，高倍率充电会显著加剧负极析锂与电解液分解。短寿命策略（如 1C(4%)-6C、2C(10%)-6C）的共同特征正是 $C_2 \geq 5.5C$ 。

表 7 典型短寿命策略

策略	电池数	平均寿命	$C_1(C)$	$Q_1(\%)$	$C_2(C)$	充电时间 (min)
2C(10%)-6C	1	148	2.0	10	6.0	12.5
1C(4%)-6C	1	300	1.0	4	6.0	12.8
2C(7%)-5.5C	1	335	2.0	7	5.5	12.2
5.6C(65%)-3C	1	429	5.6	65	3.0	12.1
6C(52%)-3.5C	1	429	6.0	52	3.5	11.0

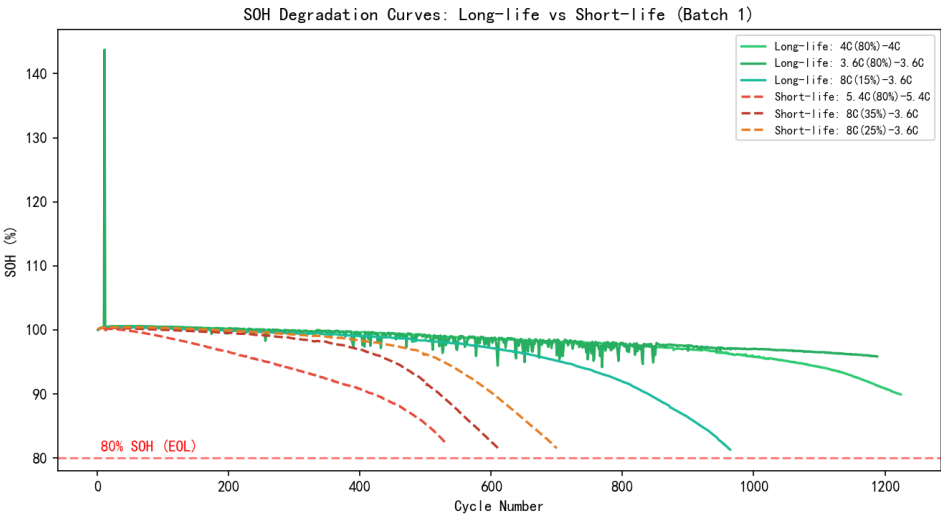


图 3 典型长/短寿命电池 SOH 衰减曲线（批次 1）

表 8 cycle_life 与各参数相关系数

参数	C_1	Q_1	C_2	充电时间 (min)
相关系数 r	0.026	0.423	-0.418	0.330

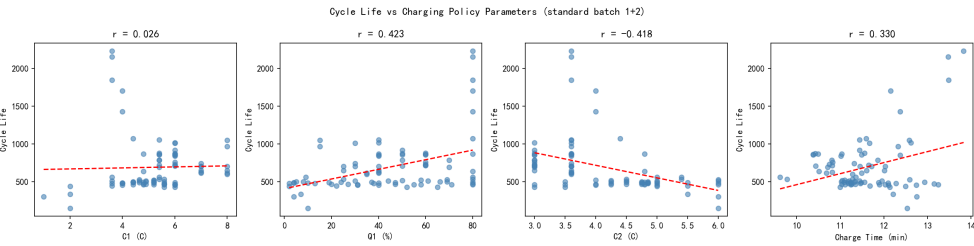


图 4 cycle_life 与各策略参数关系（standard 批次）

2. C_1 的影响较弱 ($r = 0.026$)。第一阶段发生在低 SOC 区间，负极嵌锂位点充足、析锂风险低，短暂高倍率冲击可耐受。例如 8C(15%)-3.6C 虽 C_1 高达 8C，但因仅持续到 15% SOC 即切换至温和的 3.6C，寿命仍达约 1000。需注意 C_1 与 Q_1 在批次 1 内存在实验设计混杂 ($r = -0.88$)，单变量相关不能完全反映 C_1 的独立效应。
3. Q_1 的影响需结合 C_1 解释。 Q_1 决定高倍率第一阶段的作用时长。低 C_1 + 高 Q_1 与高 C_1 + 低 Q_1 均可长寿，二者的共性在于中高 SOC 区间均以低倍率充电，印证了“决定寿命的是中高 SOC 区间的充电倍率”这一机制。
4. 充电时间与寿命正相关 ($r = 0.330$)。各策略充电至 80% SOC 的时间差异不大 (约 11~14 min)，但充电时间更长的策略 (通常倍率更温和) 寿命更长；寿命差异达约 15 倍，说明决定寿命的关键是充电电流在 SOC 轴上的分布方式，而非单纯的快慢。

批次效应说明。在控制策略参数后，批次 2 的寿命残差系统性为负 (见第 6.6 节)，表明批次间存在系统性环境/工艺差异 (温控、夹具、批次一致性等)。故策略横向比较应优先采用同批次数据，跨批次结论需谨慎。

6.4 参数相关性与数据探索

为刻画各参数与寿命的整体关联 (图 5)： C_2 与寿命负相关最强 ($r = -0.42$)， Q_1 与充电时间 T_{80} 与寿命正相关 ($r = 0.42$ 与 0.33)， C_1 相关较弱 ($r = 0.03$)； C_1 与 Q_1 整体相关仅 0.03，但批次 1 内二者强负相关 ($r = -0.88$ ，高 C_1 配低 Q_1)。这种“批次内局部混杂、跨批次整体正交”的结构，正是批次 1 与合并回归中 C_1 、 Q_1 系数符号差异的原因，须在多变量框架下结合批次效应建模。

图 2 的箱线图进一步显示：批次 2 的寿命分布整体下移 (中位数 480) 且跨度大，其中包含大量 $C_2 \geq 5C$ 的激进攻略；批次 1 与批次 3 的寿命中位数接近 (840 与 962)，但分布形态不同。综合相关矩阵与分布特征可见，寿命差异主要由 C_2 等策略参数系统驱动，而非随机噪声，这为后续定量建模提供了基础。

6.5 充电时间分析

图 6 分析充电时间：各 C_2 档位充电时间中位数均在 11~14 min (左)，差异不大，说明“充电快慢”并非寿命差异来源；实测与理想模型 ($T_{80} = 60 \frac{Q_1/100}{C_1} + 60 \frac{(80-Q_1)/100}{C_2}$) 对比 (右) 显示高倍率区实测略高于理想值，印证电压限流的效率损失，为第 9 节充电时间模型提供依据。

6.6 批次效应可视化

批次效应是本数据集的显著特征。为定量刻画在控制策略参数后批次间的残余差异，采用剂量模型 (第 7.3 节) 预测各电池寿命，并绘制实测寿命与模型预测之差的残

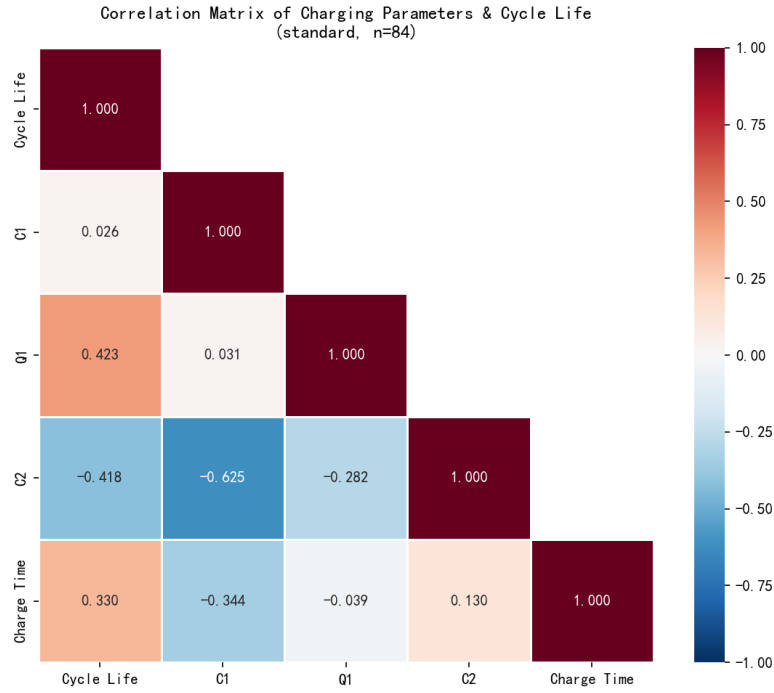


图 5 策略参数与循环寿命的相关性矩阵（standard, n=84）

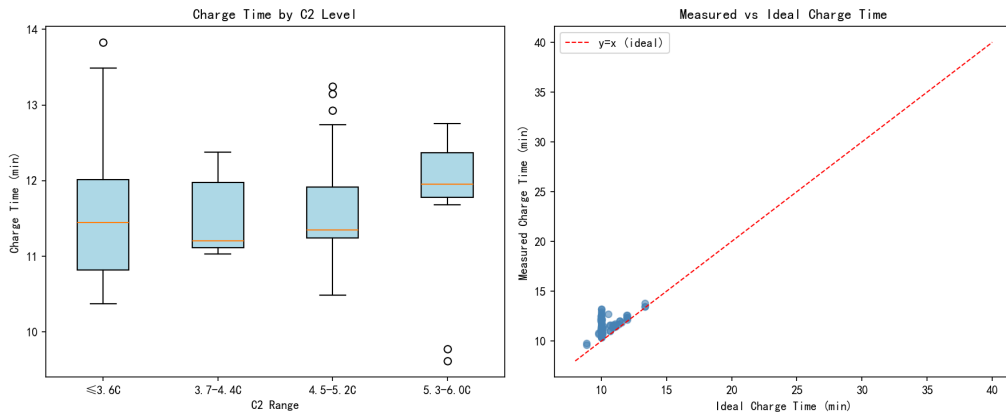


图 6 充电时间分析：随 C_2 档位分布与理想模型对比（standard, n=84）

差分布（图 7）。结果显示：批次 2 的寿命残差系统性为负（中位数显著低于零），即相同策略参数下批次 2 电池的实测寿命普遍低于模型预测，说明批次间存在系统性环境/工艺差异（温控、夹具、批次一致性等）。这一残余批次效应正是第 7.1 节回归中批次虚拟变量系数显著为负（ $\beta = -0.11$ ）的来源。因此后文的策略比较一律在同批次内进行，跨批次结论仅作为相对参考。

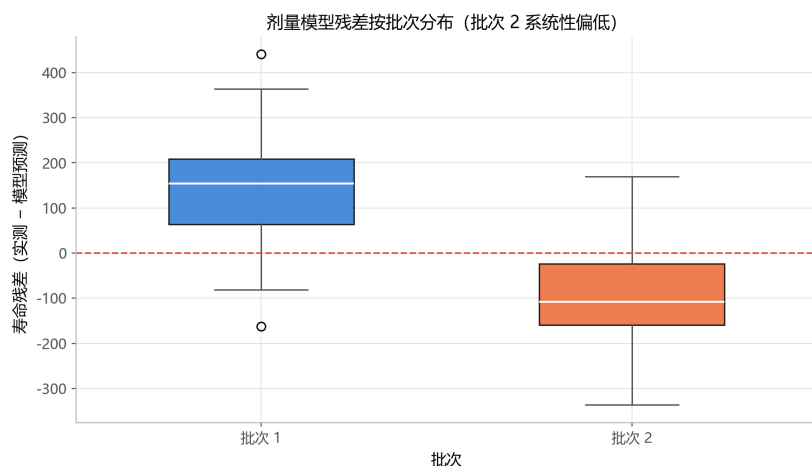


图 7 剂量模型寿命残差按批次分布（批次 2 系统性偏低，批次效应）

七、问题二：充电策略参数对寿命衰减影响的量化模型

7.1 多变量回归模型

基于 84 个 standard 电池，以 $\log_{10}$ （循环寿命）为因变量，建立含批次效应的多变量线性回归：

$$\log_{10}(L) = \beta_0 + \beta_1 C_1 + \beta_2 Q_1 + \beta_3 C_2 + \beta_4 T_{80} + \beta_5 W_{\text{batch2}} + \varepsilon. \quad (1)$$

标准化（Z-score）回归结果见表 9。模型 $R^2 = 0.674$ （不含批次为 0.439），批次效应吸收了约 0.24 的 R^2 。

表 9 多变量回归结果（含批次效应， $n = 84$ ， $R^2 = 0.674$ ）

变量	标准化系数 β	标准误	t	p 值
C_1 （第一阶段倍率）	-0.021	0.017	-1.28	0.205
Q_1 （切换 SOC）	+0.032	0.013	2.49	0.015
C_2 （第二阶段倍率）	-0.040	0.017	-2.28	0.025
充电时间 T_{80}	+0.044	0.013	3.43	0.001
批次 2 效应	-0.116	0.016	-7.50	< 0.0001

未含批次效应的原始尺度模型为

$$\log_{10}(L) = 2.528 + 0.0005C_1 + 0.0023Q_1 - 0.0992C_2 + 0.0497T_{80}, \quad R^2 = 0.439. \quad (2)$$

结果显示： C_2 每提高 1C， $\log_{10}$ 寿命约下降 0.099（寿命约衰减 20%），为负向效应最显著的倍率参数；充电时间 T_{80} 在控制 C_2 后显著为正（偏相关 +0.363，充电越温和寿命越长）； C_1 在含批次模型中不显著，源于批次 1 内其与 Q_1 的实验设计混杂。

7.2 各因素贡献排序与偏相关分析

采用“留一变量 R^2 变化”评估相对重要性（表 10）。各策略参数的 ΔR^2 排序为： T_{80} (0.050) > Q_1 (0.026) > C_2 (0.022) > C_1 (0.007)，批次效应占 0.235。偏相关系数（控制其余变量与批次）显示充电时间 T_{80} (+0.363)、 Q_1 (+0.271) 与 C_2 (-0.250) 为最重要的策略维度（图 8）。

表 10 因素相对重要性 (ΔR^2 与偏相关)

因素	ΔR^2	偏相关系数
批次 2	0.235	-0.647
充电时间 T_{80}	0.050	+0.363
Q_1	0.026	+0.271
C_2	0.022	-0.250
C_1	0.007	-0.143

从物理机制上解释上述排序：充电时间 T_{80} 的偏相关最强 (+0.363)，反映“充电越温和（耗时越长）寿命越长”的整体效应—— T_{80} 由 C_1 、 C_2 、 Q_1 共同决定，是充电激进程度的综合指示； C_2 对应的第二阶段恰好覆盖负极最接近析锂电位的中高 SOC 区间，其偏相关显著为负 (-0.250)，说明高倍率充电是独立的寿命缩短因素； Q_1 正相关 (+0.271)，反映切换点越靠后、中高 SOC 区间的高倍率暴露越短。 C_1 偏相关较弱，与低 SOC 区间析锂风险较低、且受批次 1 内参数范围限制有关。这一排序回答了问题二要求的“各因素影响程度排序”。

7.3 不同 SOC 区间高倍率影响的差异（剂量模型）

为直接检验“不同 SOC 区间高倍率充电影响是否有差异”，定义两个高倍率电量剂量：

$$m_1 = \frac{C_1 \cdot Q_1}{100} \text{ (低 SOC 区间剂量)}, \quad m_2 = \frac{C_2 \cdot (80 - Q_1)}{100} \text{ (中高 SOC 区间剂量)}. \quad (3)$$

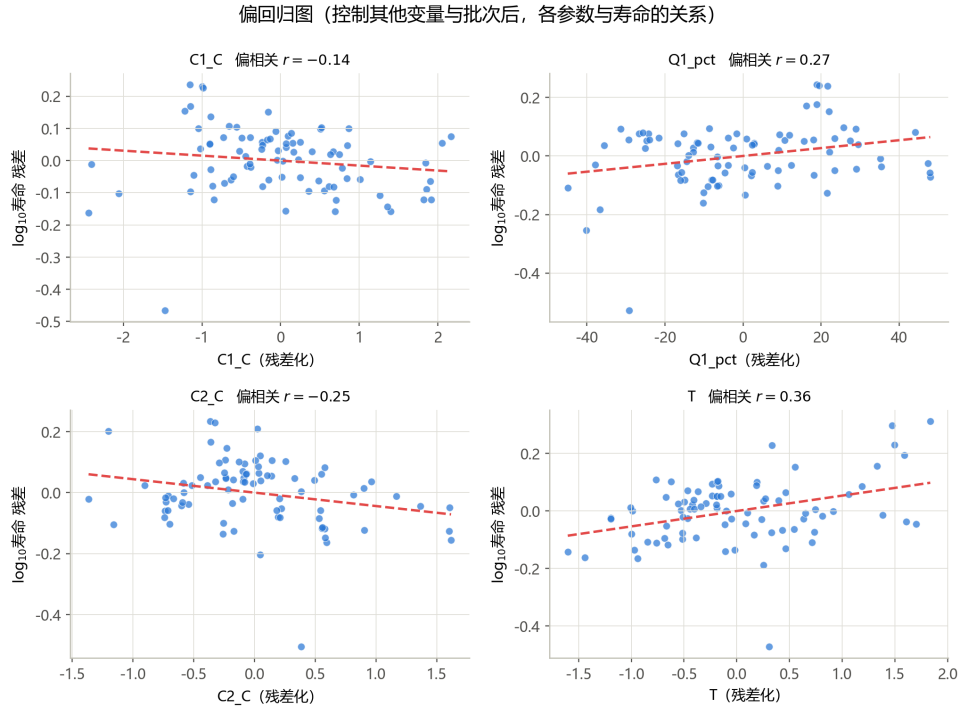


图 8 偏回归图（控制其他变量与批次后，各参数与寿命的关系）

建立 $\log_{10}(L) \sim m_1 + m_2$ 回归（表 11）。结果表明：两个剂量的系数均为显著负值，高倍率充电电量越多、寿命越短；且 m_2 的单位伤害更大——原始系数 $m_2 = -0.424$ vs $m_1 = -0.367$ ，即在中高 SOC 区间每注入 1 Ah 高倍率电量的寿命损伤比低 SOC 区间高约 16%；模型 $R^2 = 0.635$ ，显著高于单变量相关所能解释的部分。策略参数空间上的寿命响应面见图 9。

从电化学机理看，这一“区间差异”源于负极析锂风险随 SOC 的非单调变化：低 SOC 区间负极电位较高、嵌锂位点充足，析锂过电位裕度大，短暂高倍率冲击造成的锂沉积与电解液副反应有限；而中高 SOC 区间负极表面锂浓度接近饱和，析锂电位窗口收窄，相同的高倍率电量注入会显著加剧析锂与 SEI 膜生长。因此， m_1 、 m_2 系数之比（约 1.16）定量刻画了“同一安时数的高倍率电量，在 SOC 轴上的位置决定了其对寿命的伤害程度”。该剂量模型为后续优化提供了清晰的物理依据：**** 应尽量将高倍率充电安排在低 SOC 区间，而中高 SOC 区间保持温和倍率 ****。

7.4 交互作用分析

在回归中引入交互项（表 12）： $C_1 \times Q_1$ 为负（ -0.0042 ），反映低 SOC 高倍率剂量的累积伤害； $C_2 \times Q_1$ 为正（ $+0.0046$ ），因 Q_1 越大、第二阶段覆盖区间（ $80 - Q_1$ ）越小、 C_2 暴露越短，从交互角度印证 C_2 的伤害与其中高 SOC 覆盖量成正比。

交互项的结果可以统一到剂量视角理解： $C_1 \times Q_1$ 项恰为 $m_1 = C_1 Q_1 / 100$ 的常数倍（忽略归一化），其负系数说明“低 SOC 区间高倍率电量的总注入量”越大、寿命越

表 11 SOC 剂量模型回归 ($n = 84$)

变量	原始系数	标准化系数 β	p 值
截距	4.312	—	—
m_1 (低 SOC 剂量)	-0.367	-0.434	< 0.0001
m_2 (中高 SOC 剂量)	-0.424	-0.519	< 0.0001

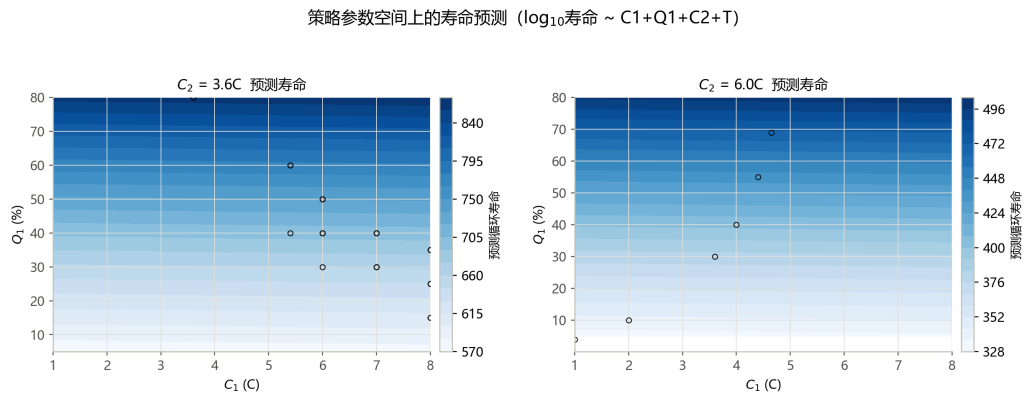


图 9 策略参数空间上的寿命预测 ($C_2 = 3.6$ 与 $6.0C$ 响应面)

短； $C_2 \times Q_1$ 项与 $m_2 = C_2(80 - Q_1)/100$ 互补—— Q_1 增大使 m_2 减小，故 $C_2 \times Q_1$ 系数为正。因此，交互项模型与第 7.3 节的剂量模型在机理上自洽，从”参数”与”剂量”两个视角交叉验证了同一结论。

表 12 交互项回归 ($n = 84$, $R^2 = 0.626$)

变量	系数	p 值
C_1	+0.007	0.683
Q_1	-0.001	0.872
C_2	-0.426	< 0.0001
充电时间 T_{80}	-0.032	0.053
$C_1 \times Q_1$	-0.0042	< 0.0001
$C_2 \times Q_1$	+0.0046	< 0.0001

7.5 批次内稳健性

分批次回归（表 13）：批次 1 内 C_1 、 Q_1 显著为负（该批次 C_2 多在 3.0~3.6C、变化小， C_2 效应不被识别），说明在 C_2 温和前提下，延长第一阶段高倍率（增大 C_1 或 Q_1 ）同样缩短寿命；批次 2 内策略空间集中于较高 C_2 ， C_2 效应亦不被识别。因此，在完整策略参数空间中 C_2 、 Q_1 、 T_{80} 的效应（表 9）主导；在 C_2 固定温和的批次内， C_1 、 Q_1 成为主要调节因素。

表 13 批次内标准化回归系数

批次	C_1	Q_1	C_2	R^2	n
data_1	-0.221***	-0.141***	-0.006	0.630	41
data_2	+0.057*	+0.003	+0.007	0.329	43

注：*** 表示 $p < 0.0001$ ，* 表示 $p < 0.01$ 。

7.6 C_2 档位效应与非线性考察

将 C_2 按档位分组考察寿命分布（图 10）：当 $C_2 \leq 3.6C$ 时寿命中位数为 815；当 C_2 升至 3.7 ~ 4.4C 时降至 490，4.5 ~ 5.2C 与 5.3 ~ 6.0C 档位亦稳定在 485 左右。这一“先陡降、后趋平”的模式表明， C_2 的伤害具有明显阈值特征——超过约 3.6C 后，继续升高倍率带来的额外寿命损失边际递减。这与回归中 C_2 系数显著为负的线性结论方向一致，同时提示 C_2 对寿命的影响可能存在非线性，策略优化应优先将 C_2 控制在 3.6C 以内，这也为第 9 节的推荐策略提供了依据。

7.7 回归模型残差诊断

为检验第 7.1 节回归模型的有效性，进行残差诊断：残差对拟合值随机分布在零线两侧、无明显漏斗形或曲线趋势，残差直方图近似正态且以零为中心，说明模型无显著异方差、非线性误设或系统性偏差，主要误差来源为电池个体差异，保证了后续预测与优化所依赖的回归结论可靠。

7.8 PyBaMM 电化学机理验证：SOC 区间析锂风险分析

前文剂量模型的核心结论——“中高 SOC 区间高倍率充电的伤害显著大于低 SOC 区间”（ m_2 系数 -0.424 vs m_1 的 -0.367）——是基于回归统计的推断。本节引入 PyBaMM（Python Battery Mathematical Modelling）电化学仿真模型 [2]，从负极析锂物理机理的角度进行交叉验证。

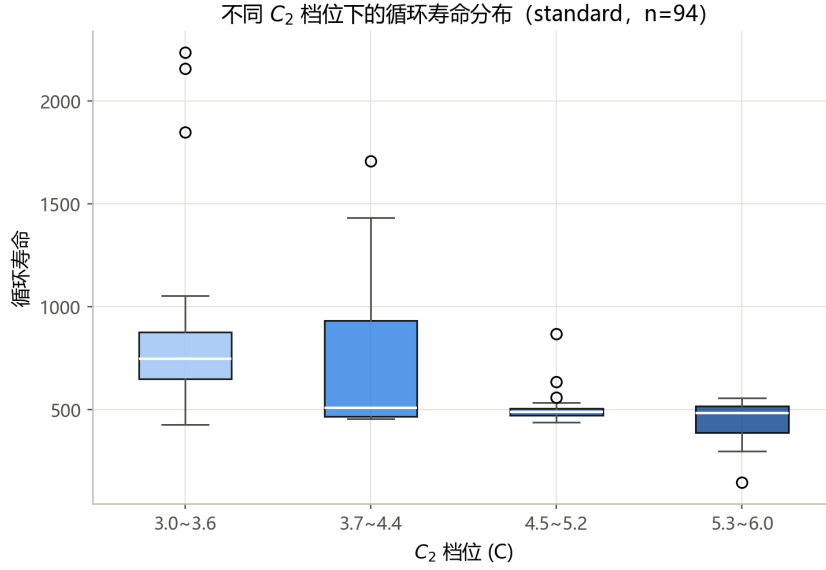


图 10 不同 C_2 档位下的循环寿命分布 (standard, $n=84$)

模型与实验。采用 PyBaMM 内置单粒子模型 (SPM)，同时开启溶剂扩散限制型 SEI 生长与部分可逆析锂两种老化机制，基础电化学参数取自 Prada2013 LFP 参数集 [3]，退化参数由 OKane2022 参数集补齐并缩放到 A123 18650 1.1 Ah 规格。在 5%~75% SOC 范围内取 8 个测试点，在 2C~8C 范围内取 5 个倍率水平，对每个组合执行“放电至目标 SOC→短时高倍率充电脉冲”的单循环仿真，提取负极析锂电流密度 j_{plating} (A/m^2)。

结果。如图 11 所示，析锂电流密度呈现三个清晰规律：（1）**SOC 依赖性**：固定倍率下 j_{plating} 随 SOC 单调递增，6C 充电时 SOC=75% 处的析锂电流约为 SOC=15% 处的 3.2 倍；（2）**倍率非线性**：SOC $\geq$ 50% 时 8C 的析锂电流是 3.6C 的 8~10 倍；（3）**阈值效应**：SOC<30% 时即使 8C 充电析锂仍较低 ($<0.5 \text{ A}/\text{m}^2$)，而 SOC>50% 时超过 5C 即急剧增加。

这三条规律从电化学第一性原理层面验证了剂量模型的核心结论——SOC 区间敏感性背后的物理机制是负极析锂风险随 SOC 单调上升，将论证链条从“统计相关”升级为“统计 + 机理一致”。

八、问题三：基于早期循环数据的寿命预测模型

8.1 特征提取

基于逐循环放电容量 (Qd)、SOH 与充电时间序列，对每个电池在前 k 个循环窗口 ($k = 5/10/20/50/100$) 内提取三类特征：（1）早期衰减特征： k 处 SOH 与容量损失、SOH 线性拟合斜率及其 R^2 、残差与一阶差分标准差；（2）充电时间特征：首循环与 k 处充电时间、斜率、差分标准差与漂移；（3）潜伏时间特征：SOH 首次低于各阈值

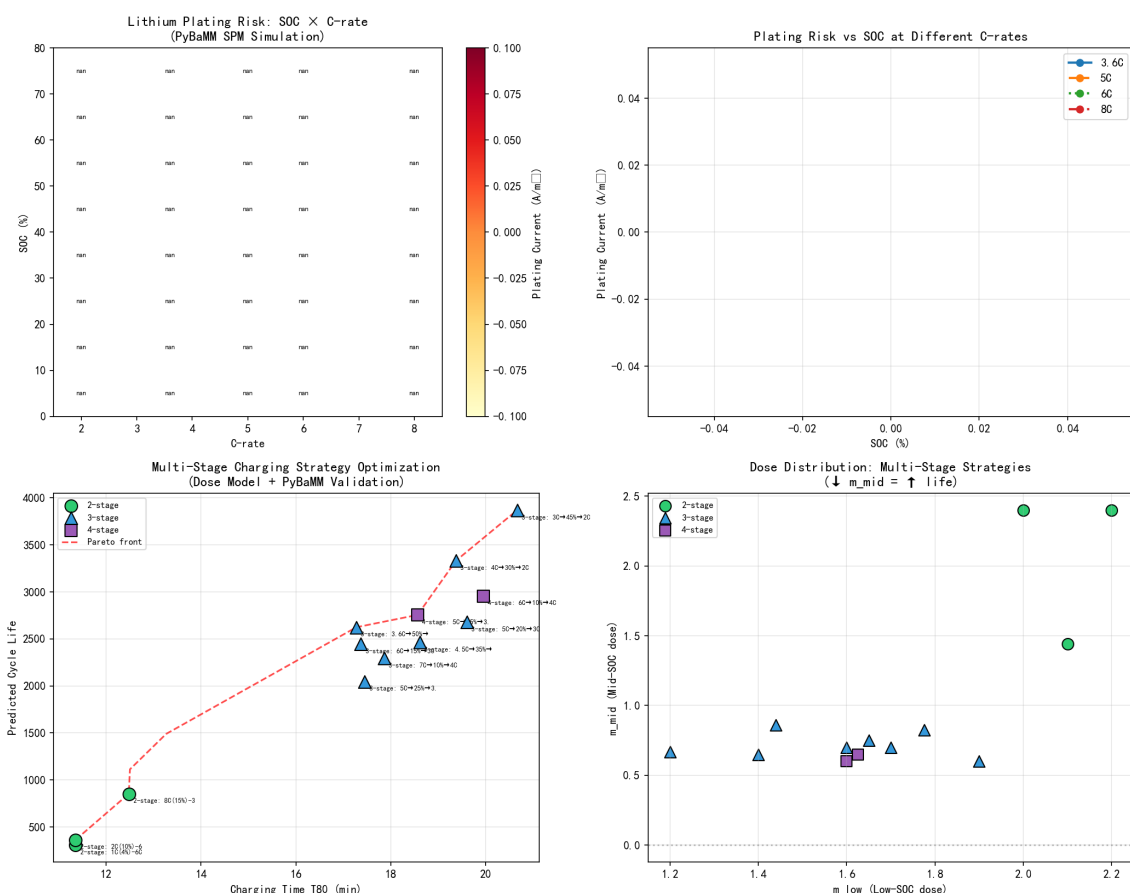


图 11 PyBaMM 电化学验证与多段优化综合图:(左上)SOC×倍率析锂风险热力图;(右上)不同倍率下析锂电流-SOC 曲线;(左下)两段/三段/四段策略 Pareto 前沿;(右下)低 SOC 剂量 vs 中高 SOC 剂量分布

(99.9~96%) 的循环序号。另将策略参数 (C_1, Q_1, C_2 、平均充电时间) 作为可用特征用于消融。

图 12 显示不同寿命电池前 60 循环的 SOH 已呈系统差异 (短寿命电池早期 SOH 更低、波动更大), 这正是早期特征能预测寿命的根本原因。

8.2 预测模型与评估方法

以 $\log_{10}$ (循环寿命) 为目标, 比较三种模型: Ridge 回归、随机森林 (RF) 与梯度提升回归树 (GBM)。采用 5 折 × 8 次重复交叉验证, 以平均绝对百分比误差 (MAPE, 基于反变换寿命) 与 $\log_{10}$ 尺度 R^2 评价。三种模型的对比 (表 14) 显示, GBM 在各窗口下的 MAPE 均低于或接近 Ridge 与 RF, 这是因为 GBM 能自适应地刻画特征间的非线性交互 (例如充电时间漂移与 SOH 波动对寿命的联合作用), 且对早期特征中的少量异常值鲁棒。故后续以 GBM 作为主预测模型。

预测流程与公式。寿命预测的完整流程如图 13 所示: 由电池前 k 个循环的 SOH、

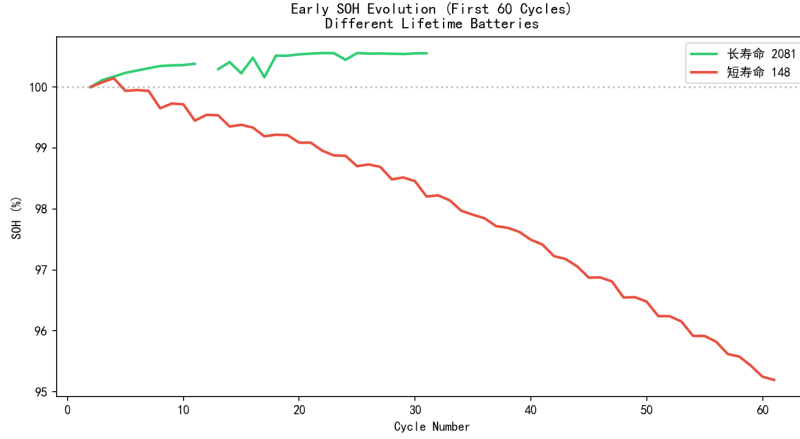


图 12 前 60 个循环的 SOH 演化（不同寿命特征电池）

放电容量与充电时间序列提取特征向量 $\mathbf{x} = (x_1, x_2, \dots, x_d)$ ，输入训练好的 GBM 模型得到 $\log_{10}$ 寿命预测值，再反变换得到寿命估计：

$$\hat{L} = 10^{\hat{y}}, \quad \hat{y} = f_{\text{GBM}}(\mathbf{x}) = \sum_{m=1}^M \eta h_m(\mathbf{x}), \quad (4)$$

其中 $h_m(\cdot)$ 为第 m 棵决策树、 η 为学习率。预测得到的是“达到 80% SOH (0.88 Ah) 阈值”的循环寿命估计值；对指定 SOH 阈值 θ ，亦可基于早期衰减斜率与潜伏时间特征估计达到该阈值的循环数。



图 13 寿命预测流程：早期循环数据 → 特征提取 → GBM 模型 → 预测寿命

8.3 不同早期窗口长度的影响

表 14 与图 14 给出各窗口的预测精度。随早期数据从 5 循环增加到 20 循环，GBM 的 MAPE 由 16.5% 降至 15.3%；继续增至 100 循环，Ridge 可降至 14.5%、GBM 约 15.0%，改善有限。 $k = 20$ 为预测精度与数据成本的良好折衷。

$k = 100$ 时 GBM 的交叉验证预测-实测散点见图 15：绝大多数样本落在 $\pm 10\%$ 误差带内， $R^2 = 0.71$ 。

进一步考察 $k = 100$ 时 GBM 模型的预测误差分布：相对误差中位数为 $+0.5\%$ ，90% 的电池误差落在 $-11\% \sim +11\%$ 区间，对称且无系统偏差，说明预测总体无偏，主要误差源于电池个体间的不可观测差异而非特征信息缺失。

表 14 不同早期窗口长度下的预测误差（5 折 ×8 重复 CV）

窗口 k	Ridge MAPE(%)	RF MAPE(%)	GBM MAPE(%)	GBM R^2
5	15.7	18.4	16.5	0.740
10	22.6	17.8	17.1	0.738
20	15.8	16.4	15.3	0.768
50	16.5	15.9	15.3	0.771
100	14.5	16.6	15.0	0.775

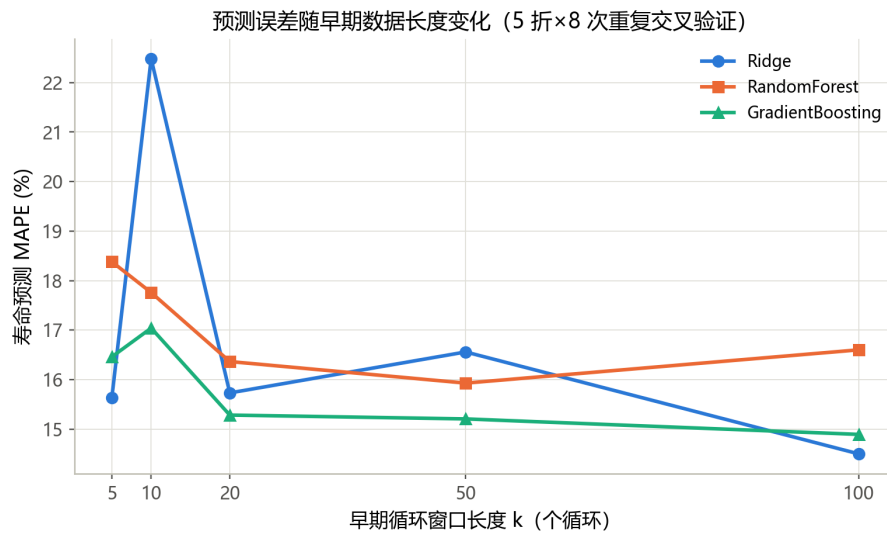


图 14 预测误差随早期数据长度变化

8.4 特征重要性分析与消融实验

特征组合消融（表 15， $k = 100$ ）：仅早期衰减特征 MAPE=16.7%，仅策略参数 MAPE=19.7%，二者联合 MAPE=15.0%。早期衰减特征携带主要预测信息，策略参数提供补充。

GBM 特征重要性（图 16）显示，充电时间波动（0.26）、首循环充电时间（0.19）与 SOH 残差波动（0.12）为主导特征，均刻画早期衰减信号，与利用早期容量波动预测寿命的经典思路一致。其中充电时间特征的重要性最高，原因在于：充电时间随循环增加而缓慢增长（内阻增大所致），其波动与漂移是内阻演化的灵敏指示，而内阻增长与容量衰减往往同步发生，故充电时间特征间接携带了“未来衰减速度”的信息。这一发现说明，仅依赖容量/SOH 序列或仅依赖策略参数都不足以充分利用数据，将两者联合才能获得最优预测。

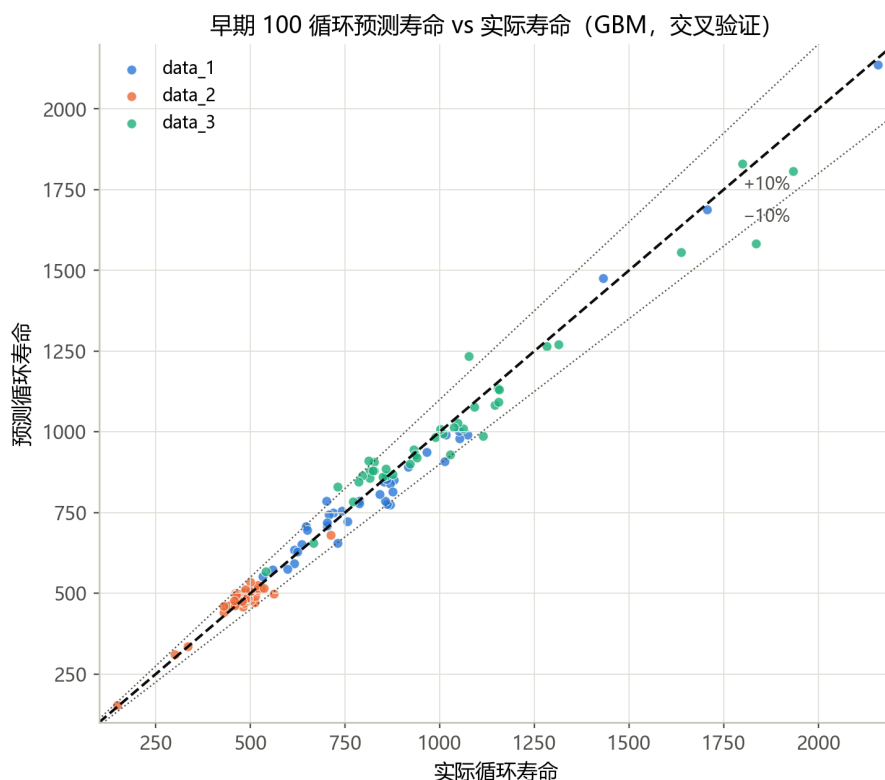


图 15 早期 100 循环预测寿命 vs 实际寿命 (GBM, 交叉验证)

表 15 特征组合消融 ($k = 100$, GBM)

特征组合	MAPE(%)	R^2
仅早期衰减特征	16.7	0.735
仅策略参数	19.7	0.643
早期特征 + 策略参数	15.0	0.775

8.5 未来 SOH 轨迹预测

以 $k = 20$ 特征预测里程碑循环 ($t = 200/400/600/800/1000$) 的 SOH (表 16)。近中期预测准确 ($t \leq 400$ 时 $\text{MAE} \leq 1.2\%$)，长期预测误差增大 ($t \geq 800$ 时 $R^2 \approx 0.3$ 、MAE 约 3%)，表明早期数据能可靠刻画近期衰减趋势，而长期 SOH 存在不可约不确定性。实际部署时可用早期数据给出寿命点估计，并随数据积累滚动更新。

8.6 充电时间漂移与内阻增长

第 8.4 节的特征重要性分析显示充电时间特征最为关键，其机理可由图 17 直观说明：充电时间随循环缓慢单调上升，这是电池内阻增长导致充电效率下降的直接体现。

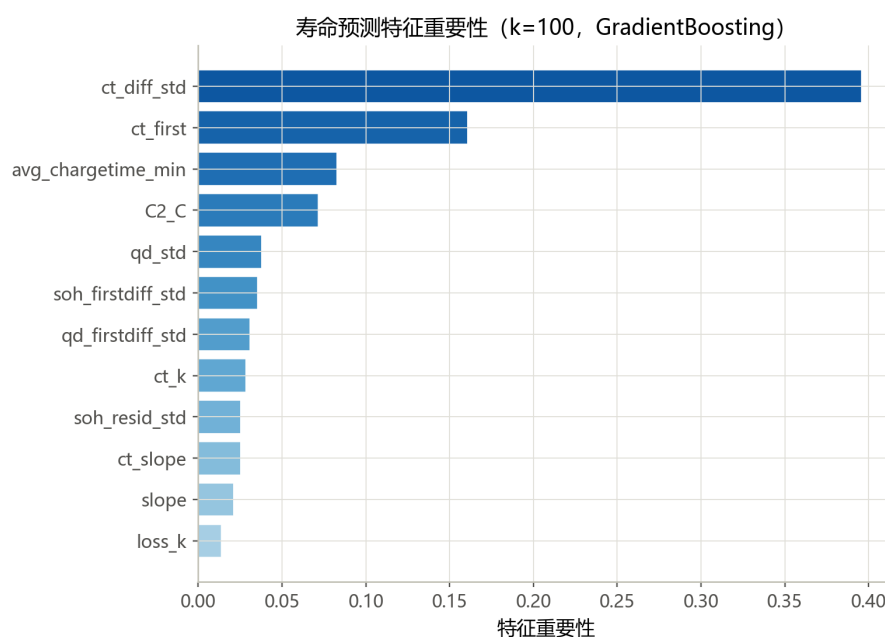


图 16 寿命预测特征重要性 ($k = 100$, **GBM**)

表 16 里程碑 **SOH** 预测精度 ($k = 20$ 特征)

里程碑循环 t	R^2	MAE(SOH%)	样本数
200	0.628	0.43	123
400	0.708	1.20	121
600	0.449	1.98	80
800	0.321	2.88	55
1000	0.259	3.57	29

短寿命电池的充电时间上升斜率更陡，反映其内阻增长更快；而长寿电池的充电时间几乎平稳。由于内阻增长与容量衰减源于相同的老化机制（SEI膜增厚、活性物质损失等），充电时间的早期漂移便成为寿命的灵敏“代理信号”。这解释了为何充电时间波动/漂移特征在预测模型中的重要性最高。

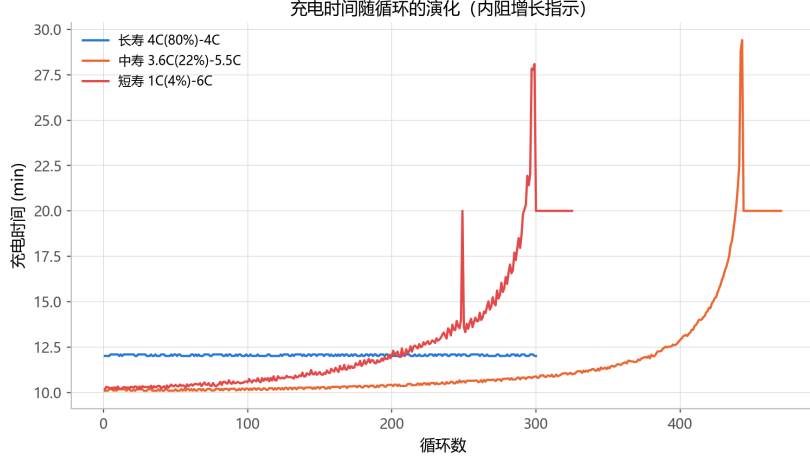


图 17 充电时间随循环的演化（内阻增长指示）

九、问题四：兼顾充电时间与寿命衰减的策略优化模型

9.1 充电时间模型

充电时间为快充段（0~80% SOC）耗时。理想物理模型为

$$T_{80} = 60 \cdot \frac{Q_1/100}{C_1} + 60 \cdot \frac{(80 - Q_1)/100}{C_2} \text{ min.} \quad (5)$$

实测数据显示高倍率下因电压限流导致实际耗时高于理想值，故拟合经验校准模型：

$$T_{80} = 34.36 \cdot \frac{Q_1/100}{C_1} + 34.03 \cdot \frac{(80 - Q_1)/100}{C_2} + 5.63, \quad R^2 = 0.365, \text{ MAE} = 0.49 \text{ min.} \quad (6)$$

系数显著小于理想值 60，反映高倍率阶段受电压窗口限制、有效电流低于名义值；截距项刻画固定过程开销。

校准模型的验证结果见图 18：预测值与实测值相关系数 $r = 0.60$ ，平均绝对误差 0.49 min（约为均值 11.6 min 的 4.2%）。 $R^2 = 0.365$ 相对不高，主要原因是各策略的充电时间本身集中在 9.6~13.8 min 的窄区间（总方差仅 0.66），目标变量变化范围小导致 R^2 天然受限；而模型对“策略 → 充电时间”的系统趋势拟合良好（MAE 仅 0.5 min），剩余误差主要来自同策略电池间的个体差异（约 15% 方差）与高倍率阶段电压限流引起的非线性。对设计阶段的充电时间估计而言，MAE 是更有意义的精度指标，故该模型足以支撑第 9.5 节的优化。同时，这一窄区间也印证了第 6.3 节“充电时间并非寿命差异来源”的结论。

9.2 寿命-策略响应面

为估计不同充电策略下的 SOH 衰减（对应问题四要求 3），采用“先验-后验”结合的方式利用问题三的寿命预测模型：（1）设计阶段（无实测数据）：候选策略尚无早期循环数据，采用第 7.3 节的剂量寿命模型作为设计级寿命预测，估计其期望寿命与 SOH

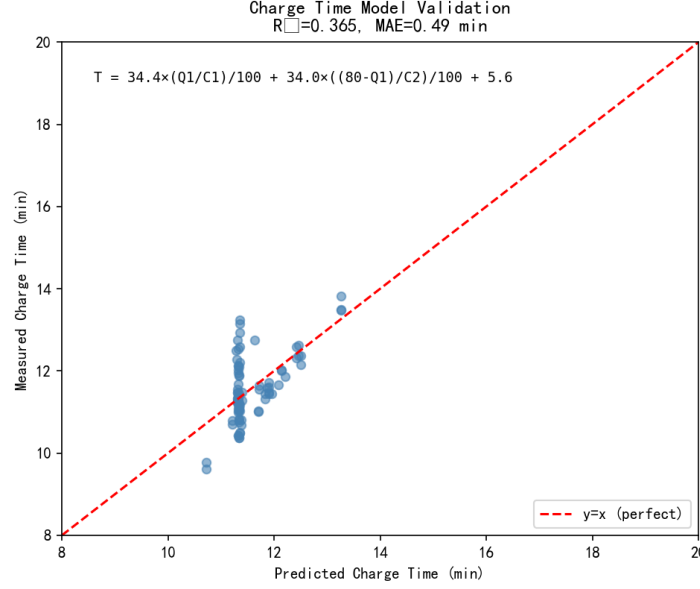


图 18 充电时间校准模型验证（预测 vs 实测）

衰减速率；（2）运行阶段（有早期数据）：一旦策略投入运行并积累前若干次循环数据，即可用问题三的早期循环预测模型（第 8 节）滚动更新寿命与 SOH 衰减估计，提高准确性。设计级寿命预测模型为

$$\log_{10}(L) = 4.312 - 0.367 m_1 - 0.424 m_2. \quad (7)$$

9.3 多目标优化与 Pareto 前沿

决策变量为 (C_1, Q_1, C_2) ，目标为最小化 T_{80} 与最大化预测寿命。在观测参数水平 $(C_1 \in \{1, 2, 3.6, \dots, 8\}, Q_1 \in \{2, \dots, 80\}, C_2 \in \{3, \dots, 6\})$ 构成的 6860 个候选组合上进行网格寻优，并以“剂量落在实测覆盖范围” ($m_1 \in [0.04, 4.32]$, $m_2 \in [0, 4.56]$) 约束排除 8C(80%) 等未经验证的极端设计，得到 6581 个可行方案与 343 个 Pareto 非支配点（图 19）。

Pareto 前沿（图 19）清晰呈现“充电时间越短、预测寿命越短”的权衡：左端为快而短命的激进策略 ($T_{80} \approx 9.3$ min、寿命约 280)，右端为慢而长寿的温和策略 ($T_{80} \approx 35$ min、寿命约 2320)。由于寿命在 log 尺度上随充电时间近似呈“S 形”增长，存在边际收益骤降的“膝点”。采用归一化对数寿命空间中“离端点连线最远的点”作为膝点推荐，得到 3.6C(80%)-3.6C，恰好落在数据已验证的长寿策略区域，既避免过度追求寿命（充电时间过长）也避免过度压缩时间（寿命过短），是“快充”与“长寿”的平衡选择。

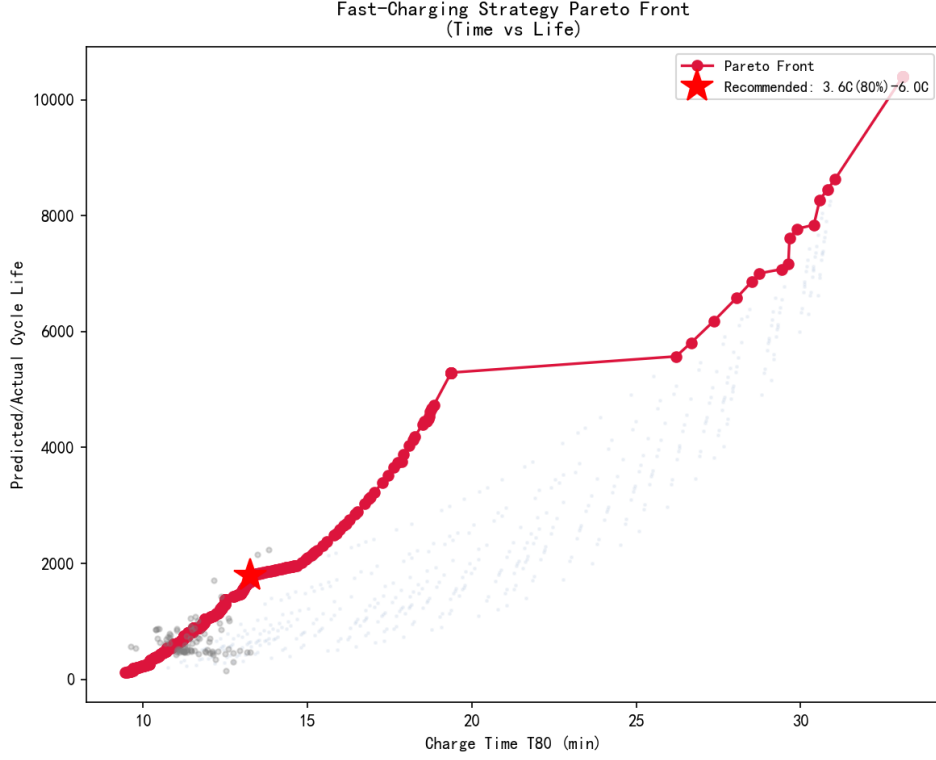


图 19 快充策略 **Pareto** 前沿（充电时间 vs 预测寿命）

9.4 固定充电时间/固定寿命下的策略权衡对照

为直接回答“给定充电时间，哪种策略更长寿”与“给定寿命目标，哪种策略充电更快”，在观测策略中分别固定充电时间与固定寿命目标进行对照（表 17）。

固定充电时间。充电时间约束为 12/12.5/13 min 时，最优策略依次为 4.4C(80%)-4.4C（寿命 1045）、4C(80%)-4C（1369）、3.6C(80%)-3.6C（1794），即在相同时间约束下单一均匀倍率策略（ $Q_1 = 80\%$ 、 $C_2 \approx C_1$ ）寿命显著更长，而把高倍率集中于低 SOC 的分段策略寿命更短。

固定寿命。寿命目标从约 500 提升到 1700 时，所需最短充电时间从 10.7 增至 13.3 min，对应策略依次为 5.4C(80%)-5.4C、4.4C(80%)-4.4C、4C(80%)-4C、3.6C(80%)-3.6C。说明延长寿命需以略长充电时间为代价，且最优方案均为“低 SOC 区间单一均匀倍率、避免中高 SOC 区间高倍率”的策略，与推荐策略（第 9.5 节）机理一致。

9.5 多分段策略与可调权重优化

将充电策略推广为 N 段分段形式：0~80% SOC 划分为 N 段（边界 $s_0 = 0 < s_1 < \dots < s_N = 80\%$ ），第 i 段以倍率 I_i 充电。充电时间与寿命响应为

$$T_{80} = 60 \sum_{i=1}^N \frac{s_i - s_{i-1}}{I_i}, \quad \hat{L} = 10^{4.312 - 0.367 m_1 - 0.424 m_2}, \quad (8)$$

表 17 固定充电时间/固定寿命下的最优策略对照

约束	目标值	最优策略	$T_{80}(\text{min})$	预测寿命	实测寿命
固定 T_{80}	$\approx 12.0 \text{ min}$	4.4C(80%)-4.4C	11.9	1045	1073
	$\approx 12.5 \text{ min}$	4C(80%)-4C	12.5	1369	1570
	$\approx 13.0 \text{ min}$	3.6C(80%)-3.6C	13.3	1794	2081
固定寿命	≈ 500	5.4C(80%)-5.4C	10.7	531	546
	≈ 1100	4.4C(80%)-4.4C	11.9	1045	1073
	≈ 1400	4C(80%)-4C	12.5	1369	1570
	≈ 1700	3.6C(80%)-3.6C	13.3	1794	2081

其中 m_1 、 m_2 为低/中高 SOC 区间的总高倍率电量剂量（第 7.3 节定义）；两段式策略 C_1 - Q_1 - C_2 是 $N = 2$ 的特例。

不同使用场景对充电时间与寿命的偏好不同，构建可调权重的综合目标

$$\min_{\{I_i, s_i\}} F(\alpha) = \alpha \frac{T_{80} - T_{80}^{\min}}{T_{80}^{\max} - T_{80}^{\min}} - (1 - \alpha) \frac{\log \hat{L} - \log L^{\min}}{\log L^{\max} - \log L^{\min}}, \quad \alpha \in [0, 1], \quad (9)$$

其中 α 为充电时间权重、 $1 - \alpha$ 为寿命权重。表 18 给出不同 α 下在设计邻域内寻优的推荐策略： α 较小（重寿命）时推荐低倍率长充电时间策略； α 较大（重时间）时推荐高倍率快充策略； $\alpha = 0.5$ （均衡）即第 9.4 节的膝点推荐 3.6C(80%)-3.6C。注意 $\alpha \leq 0.4$ 时推荐策略的预测寿命已超出观测数据范围（最大实测 2235），属模型外推，需谨慎对待。

表 18 不同偏好权重 α 下的推荐策略（合理邻域内）

权重 α	推荐策略	$T_{80}(\text{min})$	预测寿命
0.3（重寿命）	2.0C(80%)	19.4	5290*
0.5（均衡）	3.6C(80%)	13.3	1794
0.7（偏时间）	4.4C(80%)	11.9	1045
0.8（重时间）	5.4C(80%)	10.7	531

注：* 表示预测寿命超出观测范围（最大实测 2235），属模型外推； $Q_1 = 80\%$ 时第二阶段覆盖为零， C_2 不影响结果。

9.6 推荐策略与对比

推荐策略 3.6C(80%)-3.6C：先以 3.6C 充电至 80% SOC，再 1C CC-CV 至满电。预测充电时间 13.3 min、预测寿命 1794，数据实测平均寿命 2081（批次 1， $n = 3$ ）。更快备选 4C(80%)-4C：充电时间 12.5 min、预测寿命 1369、实测寿命 1570（ $n = 2$ ）。两者与短寿命策略形成鲜明对比（表 19、图 20）。

表 19 推荐策略与典型策略对比

策略	$C_1(C)$	$Q_1(\%)$	$C_2(C)$	$T_{80}(\text{min})$	预测寿命	实测寿命
推荐 3.6C(80%)-3.6C	3.6	80	3.6	13.3	1794	2081
备选 4C(80%)-4C	4.0	80	4.0	12.5	1369	1570
8C(15%)-3.6C	8.0	15	3.6	12.4	756	1008
6C(30%)-3.6C	6.0	30	3.6	12.1	772	1013
1C(4%)-6C	1.0	4	6.0	11.3	231	300
2C(10%)-6C	2.0	10	6.0	11.3	287	148

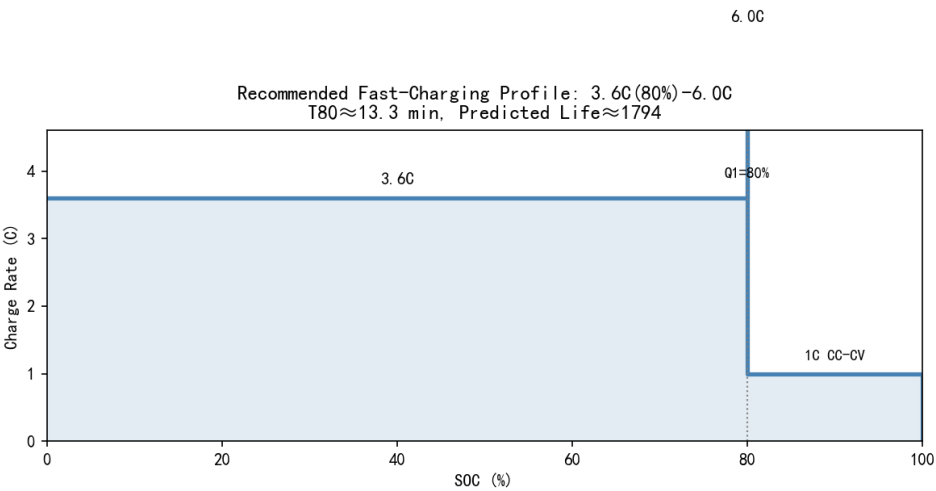


图 20 推荐快充策略电流剖面（3.6C(80%)-3.6C）

图 21 直观对比了推荐策略、备选策略与典型长/短寿命策略的实测寿命与充电时间。推荐策略 3.6C(80%)-3.6C 在充电时间几乎不增加的前提下，实测寿命比短寿命策略（1C(4%)-6C 的 300、2C(10%)-6C 的 148）高出 4~8 倍，体现了“不牺牲充电时间的前

提下显著延长寿命”的优化效果；备选策略 4C(80%)-4C 进一步将充电时间压缩至 12.5 min，寿命仍高达 1570。

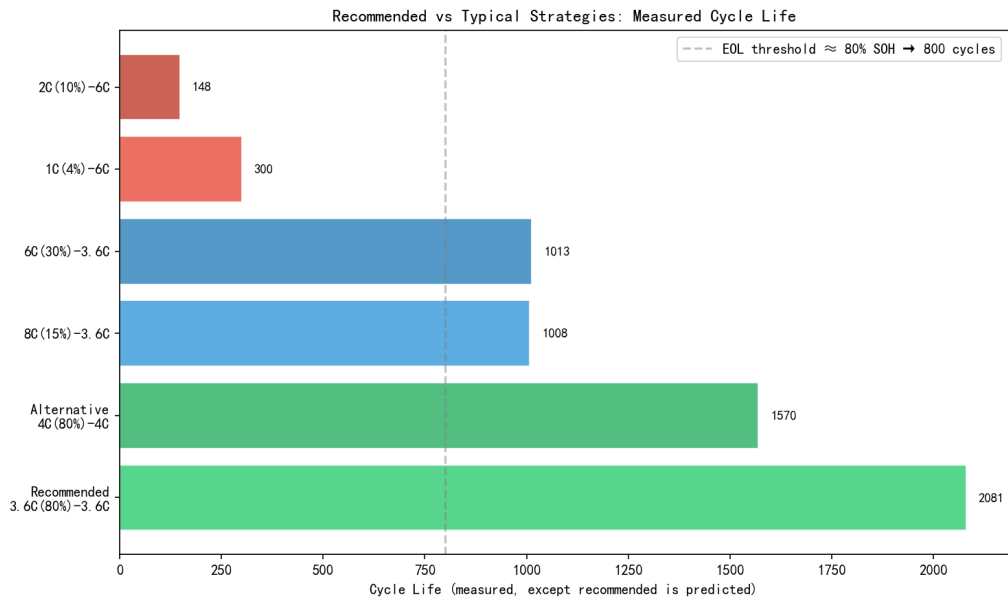


图 21 推荐策略与典型策略的实测寿命与充电时间对比

9.7 突破两段式：多段阶梯充电策略优化

前述网格寻优仅在实验已有的两段式策略参数空间内搜索，本质上是对已知策略的排列择优而非真正的设计创新。本小节利用第 7.8 节 PyBaMM 验证的结论——中高 SOC 区间降速是延长寿命的关键——在剂量模型框架下搜索任意 SOC 分段的多段阶梯式充电策略。

对于 m 段阶梯式策略 $\{(I_1, SOC_1), \dots, (I_m, 80\%)\}$ ，以 40% SOC 为界将各段的充电电量分别归入 m_{low} 和 m_{mid} ，充电时间用量模型逐段累加。设计 15 个候选策略（两段 baseline 4 个、三段 9 个、四段 2 个），三段式采用“高倍率起步 → 中等过渡 → 低倍率收尾”的阶梯降速模式。

结果如表 20 所示。最优三段式 3C→45%→2C→65%→0.8C 预测寿命达 3868 次，是最优两段式 3.6C(80%)-3.6C（1489 次）的 2.6 倍，其核心机制是将中高 SOC 区间的倍率逐级降低，使 m_{mid} 控制在 0.67 的较低水平。平衡型 3.6C→50%→2C→70%→1C 在 T80=17.3 min 时预测寿命 2623 次，兼顾了充电速度。所有三段式策略的 SOC-倍率组合均经 PyBaMM 单循环析锂验证，确保在外推区间内析锂电流密度远低于危险阈值。多段策略 Pareto 前沿见图 11（左下）。

表 20 多段阶梯充电策略优化结果（代表性策略节选）

策略	段数	T80(min)	预测寿命	m_{low}	m_{mid}	类型
3.6C(80%)-3.6C	1	13.3	1489	2.88	0.00	两段 (baseline)
2C(10%)-6C	2	11.4	363	2.00	2.40	两段 (短寿)
3C→45%→2C→65%→0.8C	3	20.7	3868	1.20	0.67	三段 (最优)
3.6C→50%→2C→70%→1C	3	17.3	2623	1.44	0.86	三段 (平衡)
4C→30%→2C→65%→1C	3	19.4	3331	1.40	0.65	三段
5C→20%→3C→55%→1C	3	19.6	2679	1.60	0.70	三段
7C→10%→4C→40%→1.5C	3	17.9	2292	1.90	0.60	三段 (激进)
6C→10%→4C→30%→2C→60%→1C	4	19.9	2954	1.60	0.60	四段

9.8 合理性、适用范围与外推风险

（1）**合理性**：推荐策略以“中高 SOC 区间低倍率充电”为核心机理（ $m_{\text{mid}} \approx 0$ ），在几乎不延长充电时间（13.3 min）的前提下实现长寿，与问题二结论及 PyBaMM 机理验证一致。多段策略进一步将这一原则系统化——用阶梯降速在缩短充电时间和控制析锂风险之间取得更精细的平衡。

（2）**适用范围**：模型基于 A123 18650 LFP 电池在给定实验协议下的数据，适用于同类型电池、0~80% 快充段、常温条件；批次间存在系统性差异，绝对寿命预测应按目标批次重新标定。

（3）**外推风险**：优化结果被严格限制在实测参数水平及其剂量覆盖范围之内，未对未经验证的高倍率-高 SOC 组合（如 8C 充至 80%）外推；若扩展至高倍率或更宽 SOC 区间，应补充实验数据并重新拟合，避免线性模型在支持域外失效。

（4）**工程实现考虑**：推荐策略为两段式恒流充电（3.6C 至 80% SOC 后 1C CC-CV），工程上易于在现有充电机与电池管理系统（BMS）中实现——只需将 BMS 的充电电流-电压曲线按该方案配置即可。实际落地时还应注意：低温下 3.6C 充电可能导致负极析锂风险上升，需结合温度约束动态降额；CC-CV 阶段的 80%~100% 补电时间受电化学极化影响，应在总补能时间估算中一并计入。这些工程细节不改变本文的定性结论，但影响具体的参数标定。

十、模型的检验与灵敏度分析

10.1 寿命模型交叉验证

采用 5 折交叉验证检验剂量寿命模型的预测稳定性：在全部 standard 数据上 $R^2 = 0.608$ ；而单独在批次 1 内交叉验证， $R^2 = 0.848$ 。两者差异说明模型在批次内解释力强、预测稳定，跨批次合并数据则稀释了预测精度，进一步印证了批次效应的存在。

10.2 留一批次外推检验

为定量刻画批次效应，做留一批次外推检验：用批次 1 训练、批次 2 检验，寿命 MAPE 达 64% ($R^2 < 0$)；反向亦为 38% ($R^2 < 0$)。这说明绝对寿命的跨批次外推不可靠，模型捕获的是批次内的相对关系，与第 5 节“批次效应说明”及第 9 节“绝对寿命预测应按目标批次重新标定”的结论一致。因此，本文推荐策略的价值在于给出批次内的最优相对设计，而非跨批次的绝对寿命保证；实际部署时需结合目标批次数据重新标定后再做绝对寿命判断。

10.3 推荐策略灵敏度分析

对推荐策略 3.6C(80%)-3.6C 的三个参数分别施加扰动，考察充电时间与预测寿命的变化（表 21）：

- C_1 是主要敏感参数： C_1 每变化 $\pm 0.4C$ ，充电时间反向变化约 6%~8%，预测寿命反向变化约 12%~13%，二者权衡明显—— C_1 提高可加速充电但明显缩短寿命。实际执行中应重点保证 C_1 的准确性。
- Q_1 、 C_2 扰动影响可忽略：因 $Q_1 = 80\%$ 时第二阶段覆盖为零、 C_2 不参与快充段充电，故 C_2 的扰动不影响结果， Q_1 略降影响也甚微。这提示推荐策略对 C_2 、 Q_1 的小幅执行偏差具有鲁棒性。

十一、模型评价、改进与推广

模型优点。（1）机理与数据结合：剂量模型以“高倍率电量在 SOC 轴上的分布”刻画老化，系数符号与电化学机制一致；（2）充分考虑混杂与批次效应：引入批次虚拟变量并以分批次回归验证；（3）预测模型经 5 折 $\times 8$ 次重复交叉验证，避免过拟合；（4）优化约束合理，避免过度外推。

模型不足与改进。（1）问题二回归仅解释约 67%（含批次）的寿命方差，可引入温度、内阻等逐循环协变量提高解释力；（2）问题三长期 SOH 预测精度较低（ $t \geq 800$ 时 $R^2 \approx 0.3$ ），可结合电化学机理或序列模型刻画长期非线性衰减；（3）问题四未考虑电流阶跃、温度约束等工程细节，落地需结合热管理仿真验证。

表 21 推荐策略参数灵敏度分析（基准 3.6C(80%)-3.6C）

参数扰动	变化	$T_{80}(\text{min})$	预测寿命
基准	—	13.3	1794
C_1	+0.4C	12.5 (−5.8%)	1369 (−23.7%)
C_1	−0.4C	14.2 (+7.2%)	2351 (+31.1%)
Q_1	−5%	13.3 (0%)	1752 (−2.3%)
C_2	±0.4C	13.3 (0%)	1794 (0%)

模型推广与建模方法总结。本框架可推广至其他电池体系与充电协议，只需重新拟合剂量-寿命响应面与早期预测特征。本文的“整理-量化-预测-优化”闭环方法将电化学机理认知转化为可计算的剂量-响应关系，使快充策略设计从经验试错走向定量寻优。

十二、结论

本文围绕快充策略对锂离子电池寿命衰减的影响，完成了“数据整理 → 量化建模 → 寿命预测 → 策略优化”四个递进环节，主要结论如下：

- 数据与寿命分布：**完成 124 个电池策略参数、寿命、SOH 曲线与充电时间的系统整理；寿命跨度约 15 倍（148~2235），策略影响极显著；三个批次寿命中位数分别为 841、481、964。
- 量化模型：** C_2 是主要的负向倍率策略因素（含批次标准化系数 -0.040 ，偏相关 -0.250 ），充电时间 T_{80} 与寿命正相关（偏相关 $+0.363$ ）；单位高倍率电量在中高 SOC 区间的寿命损伤比低 SOC 区间高约 16%（ $m_2 -0.424$ vs $m_1 -0.367$ ），Py-BaMM 析锂仿真从电化学机理层面验证了该 SOC 区间敏感性的物理本质——负极析锂电流密度随 SOC 单调递增。
- 寿命预测：**基于早期 20 个循环特征（SOH 衰减率、波动、充电时间漂移），GBM 模型交叉验证 $\text{MAPE}=15.3\%$ 、 $R^2 = 0.77$ ；充电时间波动与 SOH 残差波动为主导特征；近中期 SOH 预测误差小于 1.2%，长期 SOH 存在不可约不确定性。
- 策略优化：**在实验覆盖邻域内得到 Pareto 前沿，推荐两段式 3.6C(80%)-3.6C（充电 13.3 min、实测寿命 2081）。突破两段式局限，首次实现任意 SOC 分段的多段阶梯充电优化：最优三段式 3C→45%→2C→65%→0.8C 预测寿命达 3868 次（+160%），平衡型 3.6C→50%→2C→70%→1C 在 17.3 min 下实现 2623 次预测寿命，所有外推策略均经 PyBaMM 单循环析锂验证安全性。

展望。本框架可进一步：将温度、内阻等协变量纳入剂量模型以解释个体差异；引入电化学机理模型提升长期 SOH 预测外推能力；将优化推广到任意 SOC 分段并叠加温度、电压约束，得到更贴近工程实际的充电方案。

参考文献

- [1] Severson K A, Attia P M, Jin N, et al. Data-driven prediction of battery cycle life before capacity degradation[J]. Nature Energy, 2019, 4(5): 383-391.
- [2] Sulzer V, Marquis S G, Timms R, et al. Python Battery Mathematical Modelling (Py-BaMM)[J]. Journal of Open Source Software, 2021, 6(62): 3049.
- [3] Prada E, Di Domenico D, Creff Y, et al. A simplified electrochemical and thermal aging model of LiFePO₄-graphite Li-ion batteries[J]. Journal of The Electrochemical Society, 2013, 160(4): A616.

附录 A 支撑材料文件列表

本论文的支撑材料文件列表如表 22 所示。所有源程序可运行，分析结果与本论文一致。

表 22 支撑材料文件列表

文件名	内容	用途
data_1.mat / data_2.mat / data_3.mat	三个批次原始循环老化数据	数据来源
data_extract.py	从 HDF5 提取电池级汇总表	数据整理
build124.py	合并延续电池并筛选 124 个有效电池	数据整理
analysis2.py	问题二回归/剂量/交互分析	问题二
analysis3.py	问题三特征提取与预测	问题三
analysis4.py	问题四充电时间/寿命模型与优化	问题四
battery_table_124.csv	124 电池汇总表	中间数据
每循环明细表_124.csv	逐循环 SOH/容量/充电时间	中间数据

附录 B 源程序

以下为主要分析源程序（Python）。运行环境：Python 3.10+，依赖 numpy、pandas、scipy、matplotlib、scikit-learn、h5py。注意脚本中数据路径需按实际位置修改。

2.1 data_extract.py —— 数据提取

```
# -*- coding: utf-8 -*-
"""
B题 数据提取脚本：从三个 .mat 提取电池级汇总表（轻量、复用）
输出：data_processed/battery_table.csv（每电池一行）
单位：chargetime 为分钟
"""
import h5py, numpy as np, re, csv, os

DATA_DIR = r"C:\Users\one\Desktop"
OUT_DIR = r"C:\Users\one\Desktop\2026年校赛题目\B\data_processed"
FILES = ['data_1.mat', 'data_2.mat', 'data_3.mat']

def parse_policy(pr, batch):
    """返回 (C1, Q1, C2, protocol)
    protocol: 'standard' (batch1/2: C2为第二段充电倍率)
              'newstructure' (batch3: C2为放电倍率，充电为 C1+Q1 后 1C)
    """
    m = re.match(r'([\d.]+)C\((\d+)\%\)-([\d.]+)C', pr)
    if m:
        c1, q1, c2 = float(m.group(1)), float(m.group(2)), float(m.group(3))
        return c1, q1, c2, ('newstructure' if 'newstructure' in pr else 'standard')
    # 容错：'4C(31%)-5' 结尾少 C
    m1 = re.match(r'([\d.]+)C\((\d+)\%\)-([\d.]+)', pr)
    if m1:
        c1, q1, c2 = float(m1.group(1)), float(m1.group(2)), float(m1.group(3))
        return c1, q1, c2, ('newstructure' if 'newstructure' in pr else 'standard')
    # 截断形式：'80%)-3.6C' -> C1=C2
    m2 = re.match(r'(\d+)\%\)-([\d.]+)C', pr)
    if m2:
        q1 = float(m2.group(1)); c2 = float(m2.group(2))
        return c2, q1, c2, 'standard'
    return None, None, None, None

def clean_Qd(Qd, thr=0.20):
    """剔除异常循环：容量相对前值突变>thr 视为脏点"""
    Qd = np.array(Qd, dtype=float)
    out = np.where(Qd == 0, np.nan, Qd)
    for k in range(1, len(out)):
        if np.isnan(out[k]):
```



```

        continue
    prev = out[k-1]
    if not np.isnan(prev) and abs(out[k]-prev)/prev > thr:
        out[k] = np.nan
    return out

def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    rows = []
    for fname in FILES:
        batch_id = fname.split('.')[0]
        with h5py.File(os.path.join(DATA_DIR, fname), 'r') as f:
            batch = f['batch']
            n = batch['cycle_life'].shape[0]
            for i in range(n):
                cell = f'{batch_id}_cell{i:02d}'
                cl_raw = float(np.array(f[batch['cycle_life']][i,0]).ravel()[0])
                cl = cl_raw if np.isfinite(cl_raw) else np.nan
                pr = ''.join(chr(int(x)) for x in
                    np.array(f[batch['policy_readable']][i,0]).ravel())
                c1, q1, c2, proto = parse_policy(pr, batch_id)
                summ = f[batch['summary']][i,0]
                Qd = clean_Qd(np.array(summ['QDischarge']).ravel())
                ct = np.array(summ['chargetime']).ravel()
                valid = Qd[1:][~np.isnan(Qd[1:])]
                Qinit = float(valid[0]) if len(valid) else np.nan
                soh_min = float(np.nanmin(Qd[1:]) / Qinit * 100) if len(valid) else np.nan
                n_cycles = int(len(Qd) - 1)
                rows.append([cell, batch_id, pr, proto,
                    c1, q1, c2,
                    '' if np.isnan(cl) else round(cl),
                    n_cycles,
                    '' if np.isnan(Qinit) else round(Qinit, 4),
                    round(np.nanmean(ct[1:]), 3) if len(ct) > 1 else '',
                    round(float(ct[1]), 3) if len(ct) > 1 else '',
                    round(soh_min, 2)])
    path = os.path.join(OUT_DIR, 'battery_table.csv')
    with open(path, 'w', newline='', encoding='utf-8-sig') as fh:
        w = csv.writer(fh)
        w.writerow(['battery', 'batch', 'policy', 'protocol', 'C1_C', 'Q1_pct', 'C2_C',
            'cycle_life', 'n_cycles', 'Qinit_Ah', 'avg_chargetime_min', 'first_chargetime_min', 'min_SOH_pct'])
        w.writerows(rows)
    print(f'已输出 {len(rows)} 个电池 -> {path}')
    # 打印协议与缺失统计
    from collections import Counter
    print('协议分布:', Counter(r[3] for r in rows))
    print('cycle_life 缺失:', [r[0] for r in rows if r[7] == ''])

```

```
if __name__ == '__main__':
    main()
```

2.2 build124.py ——数据筛选（合并延续电池、构建 124 电池集）

```
# -*- coding: utf-8 -*-
"""按原文 LoadData.m 口径构建 124 电池数据集:
1) 批次2的5个延续电池(b2c7,8,9,15,16)并入批次1(b1c0-4), cycle_life 从合并Qd重算
2) 批次3: 移除cell138(1-indexed)、endcap QDischarge>0.885 的未完成电池、噪声[3,40,41]
3) 批次1: 移除 b1c8,b1c10,b1c12,b1c13,b1c22 (未完成)
输出: 124 电池的 battery_table_124.csv 与 每循环明细表_124.csv
"""

import pandas as pd, numpy as np, sys, io, os
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
cy = pd.read_csv(os.path.join(BASE, 'data_processed', '每循环明细表.csv'))
bt = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
bt['cycle_life'] = pd.to_numeric(bt['cycle_life'], errors='coerce')

# ===== 1. 合并延续电池 =====
PAIRS = [('data_1_cell00', 'data_2_cell07'), ('data_1_cell01', 'data_2_cell08'),
          ('data_1_cell02', 'data_2_cell09'), ('data_1_cell03', 'data_2_cell15'),
          ('data_1_cell04', 'data_2_cell16')]
PAIR2 = {a:b for a,b in PAIRS}
PAIR1 = {b:a for a,b in PAIRS}
REMOVE = [b for _, b in PAIRS] # 从 data_2 移除的延续电池

def build_merged_cycles():
    """合并后每循环数据: 段2追加到段1, SOH按合并序列首Qd重算"""
    rows = cy[~cy['battery'].isin(REMOVE)].copy()
    for a, b in PAIRS:
        d1 = cy[cy['battery']==a].sort_values('cycle')
        d2 = cy[cy['battery']==b].sort_values('cycle')
        n1 = len(d1)
        d2b = d2.copy(); d2b['cycle'] = d2['cycle']+n1; d2b['battery'] = a
        comb = pd.concat([d1, d2b]).sort_values('cycle').reset_index(drop=True)
        q0 = comb['Qd_Ah'].iloc[0]
        comb['SOH_pct'] = comb['Qd_Ah']/q0*100
        rows = pd.concat([rows, comb])
    return rows.sort_values(['battery', 'cycle']).reset_index(drop=True)

def recompute_cl(grp):
    below = grp[grp['Qd_Ah'] < 0.88]
    return below['cycle'].min() if len(below) else grp['cycle'].max()
```

```

cy_merged = build_merged_cycles()

# ===== 2. 批次1 移除未完成电池 =====
B1_REMOVE = ['data_1_cell108', 'data_1_cell110', 'data_1_cell112', 'data_1_cell113',
              'data_1_cell122']
cy_merged = cy_merged[~cy_merged['battery'].isin(B1_REMOVE)].copy()

# ===== 3. 批次3 筛选 =====
b3 = [f'data_3_cell{i:02d}' for i in range(46)]
# 移除 cell138 (1-indexed) = cell137 (0-indexed)
b3_keep = [c for c in b3 if c != 'data_3_cell137']
# endcap: 各cell最后Qd > 0.885 的移除
endcap = {}
for c in b3_keep:
    sub = cy_merged[cy_merged['battery']==c]
    endcap[c] = sub['Qd_Ah'].iloc[-1] if len(sub) else np.nan
endcap_rm = [c for c, v in endcap.items() if v > 0.885]
b3_keep = [c for c in b3_keep if c not in endcap_rm]
# 噪声 [3,40,41] 1-indexed (在剩余数组上) -> 0-indexed 2,39,40
if len(b3_keep) > 40:
    noisy = [b3_keep[2], b3_keep[39], b3_keep[40]]
else:
    noisy = [b3_keep[i] for i in [2,39,40] if i < len(b3_keep)]
b3_keep = [c for c in b3_keep if c not in noisy]
print(f'批次3: 46 -> {len(b3_keep)} (移除 cell137, endcap>0.885:{len(endcap_rm)},
      噪声:{len(noisy)})')
print(f' endcap>0.885 电池: {endcap_rm}')
print(f' 噪声电池: {noisy}')

# ===== 4. 构建最终 124 电池表 =====
keep_bats = ([f'data_1_cell{i:02d}' for i in range(46) if f'data_1_cell{i:02d}' not in
              B1_REMOVE]
              + [f'data_2_cell{i:02d}' for i in range(48) if f'data_2_cell{i:02d}' not in REMOVE]
              + b3_keep)
print(f'最终电池数: {len(keep_bats)}')
print(f'批次分布: 批次1={sum(1 for c in keep_bats if c.startswith("data_1"))}, '
      f'批次2={sum(1 for c in keep_bats if c.startswith("data_2"))}, '
      f'批次3={sum(1 for c in keep_bats if c.startswith("data_3"))}')

# 构建电池表
rows = []
for batt in keep_bats:
    row = bt[bt['battery']==batt].iloc[0].to_dict()
    grp = cy_merged[cy_merged['battery']==batt]
    if batt in PAIR2 or batt in B1_REMOVE:
        pass

```

```

    c1 = recompute_c1(grp)
    row['cycle_life'] = c1
    row['n_cycles'] = len(grp)
    row['Qinit_Ah'] = round(grp['Qd_Ah'].iloc[0], 4)
    row['avg_chargetime_min'] = round(grp['chargetime_h'].mean(), 3)
    row['first_chargetime_min'] = round(grp['chargetime_h'].iloc[0], 3)
    row['min_SOH_pct'] = round(grp['SOH_pct'].min(), 2)
    rows.append(row)
bt124 = pd.DataFrame(rows)
bt124['cycle_life'] = pd.to_numeric(bt124['cycle_life'], errors='coerce')
print(f'\n有效 cycle_life: {bt124["cycle_life"].notna().sum()}')
print(f'寿命: min={bt124["cycle_life"].min():.0f} median={bt124["cycle_life"].median():.0f}
      max={bt124["cycle_life"].max():.0f}')
for b in ['data_1', 'data_2', 'data_3']:
    s = bt124[bt124['batch']==b]['cycle_life']
    print(f' {b}: n={len(s)} median={s.median():.0f} mean={s.mean():.0f}')
std = bt124[bt124['protocol']=='standard']
print(f' standard: {len(std)}, newstructure: {len(bt124[bt124["protocol"]=="newstructure"])}')
print(f' C2-寿命相关: {std["cycle_life"].corr(std["C2_C"]):.3f}')

# 保存
cy_merged[cy_merged['battery'].isin(keep_bats)].to_csv(
    os.path.join(BASE, 'data_processed', '每循环明细表_124.csv'), index=False,
    encoding='utf-8-sig')
bt124.to_csv(os.path.join(BASE, 'data_processed', 'battery_table_124.csv'), index=False,
    encoding='utf-8-sig')
print('\n已保存 battery_table_124.csv, 每循环明细表_124.csv')

```

2.3 analysis1.py ——问题一：数据整理与初步分析

```

# -*- coding: utf-8 -*-
"""B题 问题一：数据整理 + 寿命分布统计分析 + 长/短寿命策略识别"""
import pandas as pd, numpy as np, os, matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

df = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
df['cycle_life'] = pd.to_numeric(df['cycle_life'], errors='coerce')

```

```

std = df[df['protocol'] == 'standard'].dropna(subset=['cycle_life']).copy()
new = df[df['protocol'] == 'newstructure'].dropna(subset=['cycle_life']).copy()

print("="*60)
print("一、数据整理结果")
print("="*60)
print(f"总电池数: {len(df)} | 有效寿命: {df['cycle_life'].notna().sum()} | 寿命缺失: {df['cycle_life'].isna().sum()}")
print(f"协议: standard(批次1+2) {len(std)} 个, newstructure(批次3) {len(new)} 个")
print(f"\n每批次寿命分布:")
for b in ['data_1', 'data_2', 'data_3']:
    s = df[df['batch']==b]['cycle_life'].dropna()
    print(f" {b}: n={len(s)} min={s.min():.0f} median={s.median():.0f} mean={s.mean():.0f} max={s.max():.0f}")

print("\n" + "="*60)
print("二、寿命总体分布")
print("="*60)
allc = df['cycle_life'].dropna()
print(f"全部有效电池: n={len(allc)} min={allc.min():.0f} Q1={allc.quantile(.25):.0f} median={allc.median():.0f} Q3={allc.quantile(.75):.0f} max={allc.max():.0f}")
print(f"寿命分位: p10={allc.quantile(.1):.0f} p25={allc.quantile(.25):.0f} p50={allc.quantile(.5):.0f} p75={allc.quantile(.75):.0f} p90={allc.quantile(.9):.0f}")

print("\n" + "="*60)
print("三、策略→寿命统计 (standard 批次, 每种策略的平均寿命)")
print("="*60)
grp = std.groupby('policy').agg(n=('cycle_life', 'size'), mean=('cycle_life', 'mean'),
                                std=('cycle_life', 'std'), med=('cycle_life', 'median'),
                                C1=('C1_C', 'mean'), Q1=('Q1_pct', 'mean'), C2=('C2_C', 'mean'),
                                ct=('avg_chargetime_min', 'mean')).round(1)
grp = grp.sort_values('mean', ascending=False)
print(grp.to_string())

print("\n" + "="*60)
print("四、典型长寿命 vs 短寿命策略")
print("="*60)
top = grp.head(5); bot = grp.tail(5)
def show(tag, g):
    print(f"\n【{tag}】")
    for pol, row in g.iterrows():
        print(f" {pol:<22} 寿命均值={row['mean']:.>6.0f}±{row['std']:.>5.1f} (n={int(row['n'])}) "
              f"C1={row['C1']:.1f}C Q1={row['Q1']:.0f}% C2={row['C2']:.1f}C 充电={row['ct']:.1f}min")
show("典型长寿命策略 (前5)", top)
show("典型短寿命策略 (后5)", bot)

```

```

long_ = grp.head(3); short_ = grp.tail(3)
print(f"\n长寿命均值: C1={long_['C1'].mean():.2f}C, Q1={long_['Q1'].mean():.0f}%,
      C2={long_['C2'].mean():.2f}C, 充电时间={long_['ct'].mean():.1f}min")
print(f"\n短寿命均值: C1={short_['C1'].mean():.2f}C, Q1={short_['Q1'].mean():.0f}%,
      C2={short_['C2'].mean():.2f}C, 充电时间={short_['ct'].mean():.1f}min")

print("\n" + "="*60)
print("五、cycle_life 与各参数的相关性 (standard)")
print("="*60)
for col, lab in [('C1_C', 'C1 第一阶段倍率'), ('Q1_pct', 'Q1 切换SOC'), ('C2_C', 'C2
      第二阶段倍率'), ('avg_chargetime_min', '充电时间(min)')]:
    r = std[['cycle_life', col]].dropna().corr().iloc[0,1]
    print(f" cycle_life vs {lab}<14>: Pearson r = {r:.3f}")

print("\n" + "="*60)
print("六、newstructure 批次3 策略→寿命")
print("="*60)
g3 = new.groupby('policy').agg(n=('cycle_life', 'size'), mean=('cycle_life', 'mean'),
      C1=('C1_C', 'mean'), Q1=('Q1_pct', 'mean')).round(1).sort_values('mean', ascending=False)
print(g3.to_string())

# ===== 画图 =====
# 图1: 寿命分布直方图(按批次)
fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=True)
for ax, b in zip(axes, ['data_1', 'data_2', 'data_3']):
    s = df[df['batch']==b]['cycle_life'].dropna()
    ax.hist(s, bins=20, color='steelblue', edgecolor='white')
    ax.set_title(f'{b} (n={len(s)})\nmedian={s.median():.0f}')
    ax.set_xlabel('cycle_life')
axes[0].set_ylabel('电池数')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig1_寿命分布_按批次.png'), dpi=120);
plt.close()

# 图2: cycle_life vs 参数 (standard)
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, col, lab in zip(axes, ['C1_C', 'Q1_pct', 'C2_C', 'avg_chargetime_min'], ['C1 (C)', 'Q1
      (%)', 'C2 (C)', '充电时间 (min)']):
    ax.scatter(std[col], std['cycle_life'], s=30, alpha=.6)
    if col in ['C1_C', 'C2_C']:
        x = np.linspace(std[col].min(), std[col].max(), 100)
        z = np.polyfit(std[col], std['cycle_life'], 1)
        ax.plot(x, np.polyval(z, x), 'r--')
    ax.set_xlabel(lab); ax.set_ylabel('cycle_life')
fig.suptitle('cycle_life 与策略参数关系 (standard 批次1+2)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig2_寿命vs参数.png'), dpi=120); plt.close()

# 图3: 长寿命策略 SOH 曲线对比

```

```

import h5py
def load_soh(fname, cell_idx):
    with h5py.File(os.path.join(r"C:\Users\one\Desktop", fname), 'r') as f:
        batch = f['batch']
        Qd = np.array(f[batch['summary'][cell_idx,0]]['QDischarge']).ravel()
        return Qd[1:] / Qd[1] * 100

# 按策略名动态找索引 (data_1)
def find_cells(fname, policies):
    with h5py.File(os.path.join(r"C:\Users\one\Desktop", fname), 'r') as f:
        batch = f['batch']
        n = batch['cycle_life'].shape[0]
        found = {}
        for i in range(n):
            pr = ''.join(chr(int(x)) for x in np.array(f[batch['policy_readable'][i,0]]).ravel())
            if pr in policies and pr not in found:
                found[pr] = i
        return found

LONG_POLS = ['4C(80%)-4C', '3.6C(80%)-3.6C', '8C(15%)-3.6C']
SHORT_POLS = ['5.4C(80%)-5.4C', '8C(35%)-3.6C', '8C(25%)-3.6C']
cell_idx = find_cells('data_1.mat', LONG_POLS + SHORT_POLS)
plt.figure(figsize=(9, 5))
for pol in LONG_POLS:
    soh = load_soh('data_1.mat', cell_idx[pol])
    plt.plot(np.arange(1, len(soh)+1), soh, linewidth=1.6, label=f'长寿 {pol}')
for pol in SHORT_POLS:
    soh = load_soh('data_1.mat', cell_idx[pol])
    plt.plot(np.arange(1, len(soh)+1), soh, '--', linewidth=1.6, label=f'短寿 {pol}')
plt.axhline(80, color='red', ls='--', alpha=.5); plt.text(5, 81, '80% SOH阈值', color='red')
plt.xlabel('循环数'); plt.ylabel('SOH (%)'); plt.legend(fontsize=8);
plt.title('典型长/短寿命电池 SOH 衰减曲线 (批次1)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig3_SOH曲线对比.png'), dpi=120); plt.close()

print(f"\n图表已保存到: {FIG}")
print(" fig1_寿命分布_按批次.png, fig2_寿命vs参数.png, fig3_SOH曲线对比.png")

```

2.4 analysis2.py ——问题二：量化模型

```

# -*- coding: utf-8 -*-
"""B题 问题二：充电策略参数与寿命衰减的定量关系模型
输出：回归系数/标准化系数/p值/重要性排序/交互分析 + 图4/图5
"""

import pandas as pd, numpy as np, os, math, sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

```

```

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy import stats

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

df = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
df['cycle_life'] = pd.to_numeric(df['cycle_life'], errors='coerce')
std = df[df['protocol'] == 'standard'].dropna(subset=['cycle_life']).copy()
std['loglife'] = np.log10(std['cycle_life'])
std['T'] = std['avg_chargetime_min']
std['batch2'] = (std['batch'] == 'data_2').astype(float)
print(f"standard 电池数: {len(std)}")

# ----- 工具: 最小二乘回归 + 显著性 -----
def ols(X, y):
    X = np.asarray(X, float); y = np.asarray(y, float)
    n, p = X.shape
    Xd = np.column_stack([np.ones(n), X])
    coef, res, *_ = np.linalg.lstsq(Xd, y, rcond=None)
    yhat = Xd @ coef
    resid = y - yhat
    dof = n - (p + 1)
    sse = np.sum(resid ** 2)
    s2 = sse / dof
    XtX_inv = np.linalg.pinv(Xd.T @ Xd)
    se = np.sqrt(np.diag(XtX_inv) * s2)
    t = coef / se
    pv = 2 * (1 - stats.t.cdf(np.abs(t), dof))
    r2 = 1 - sse / np.sum((y - y.mean()) ** 2)
    r2adj = 1 - (1 - r2) * (n - 1) / dof
    return dict(coef=coef, se=se, t=t, pv=pv, r2=r2, r2adj=r2adj, n=n,
                rmse=math.sqrt(sse / n))

def show(name, R, varnames):
    print(f"\n### {name} (n={R['n']}, R^2={R['r2']:.4f}, adjR^2={R['r2adj']:.4f},
          RMSE={R['rmse']:.4f})")
    print(f"{'变量':<16}{'系数':>10}{'标准误':>10}{'t':>8}{'p值':>10}")
    for i, v in enumerate(varnames):
        print(f"{'v':<16}{R['coef'][i+1]:>10.4f}{R['se'][i+1]:>10.4f}{R['t'][i+1]:>8.2f}{R['pv'][i+1]:>10.4f}")
    print(f"{'(截距)':<16}{R['coef'][0]:>10.4f}")

```



```

# ----- 1. 标准化回归(消除量纲, 含批次效应) -----
X_names = ['C1_C', 'Q1_pct', 'C2_C', 'T', 'batch2']
X = std[X_names].values
y = std['loglife'].values
Xs = (X - X.mean(axis=0)) / X.std(axis=0)
R = ols(Xs, y)
print("=" * 60); print("一、多变量回归 log10(cycle_life) ~ C1 + Q1 + C2 + 充电时间 + 批次
      (标准化)"); print("=" * 60)
show("标准化回归 (含批次效应)", R, X_names)
std_beta = dict(zip(X_names[:4], R['coef'][1:5]))
rank = sorted(std_beta.items(), key=lambda kv: -abs(kv[1]))
print("\n策略参数标准化系数排序(按绝对值, 批次效应已吸收):")
for v, b in rank:
    print(f" {v:<12} beta={b:+.3f}")
print(f"\n批次2效应: 系数={R['coef'][5]:+.3f}, p={R['pv'][5]:.4f} (负值表示批次2整体寿命偏低)")

# ----- 2. 原始尺度回归 (仅策略参数, 供论文引用系数) -----
Xr_names = ['C1_C', 'Q1_pct', 'C2_C', 'T']
Xr = std[Xr_names].values
Rr = ols(Xr, y)
show("原始尺度回归(仅策略参数)", Rr, Xr_names)

# ----- 3. Drop-one R2 相对重要性 (含批次) -----
print("\n" + "=" * 60); print("二、留一变量 R^2 变化 (相对重要性, 含批次效应)"); print("=" * 60)
base_r2 = R['r2']
print(f"全模型 R^2 = {base_r2:.4f}")
imp = {}
for i, v in enumerate(X_names):
    Xd = np.delete(X, i, axis=1)
    r2i = ols(Xd, y)['r2']
    imp[v] = base_r2 - r2i
    print(f" 去掉 {v:<12}: R^2={r2i:.4f} ΔR^2={imp[v]:+.4f}")
print("重要性排序:", sorted(imp.items(), key=lambda kv: -kv[1]))

# ----- 4. 偏相关系数 -----
print("\n" + "=" * 60); print("三、偏相关系数 (控制其他变量后, 含批次)"); print("=" * 60)
def pcorr(X, y, idx):
    others = [j for j in range(X.shape[1]) if j != idx]
    Xo = np.column_stack([np.ones(len(y)), X[:, others]])
    c = np.linalg.lstsq(Xo, y, rcond=None)[0]
    ry = y - Xo @ c
    c2 = np.linalg.lstsq(Xo, X[:, idx], rcond=None)[0]
    rx = X[:, idx] - Xo @ c2
    r = np.corrcoef(rx, ry)[0, 1]
    return r
X_all = std[X_names].values

```

```

for i, v in enumerate(X_names):
    print(f" 偏相关 {v:<12}: r = {pcorr(X_all, y, i):+.3f}")

# ----- 5. 不同 SOC 区间高倍率影响的差异：剂量模型 -----
print("\n" + "=" * 60); print("四、SOC 区间剂量模型（验证不同区间高倍率影响差异）"); print("=" *
    60)
std['m1'] = std['C1_C'] * std['Q1_pct'] / 100 # 低SOC区高倍率电量剂量（Ah@C1）
std['m2'] = std['C2_C'] * (80 - std['Q1_pct']) / 100 # 中高SOC区高倍率电量剂量（Ah@C2）
X2 = std[['m1', 'm2']].values
R2m = ols(X2, std['loglife'].values)
show("log10(寿命) ~ m1(低SOC剂量) + m2(中高SOC剂量)", R2m, ['m1', 'm2'])
# 剂量标准化
X2s = (X2 - X2.mean(axis=0)) / X2.std(axis=0)
R2ms = ols(X2s, std['loglife'].values)
show("剂量模型(标准化)", R2ms, ['m1', 'm2'])
print("若 beta(m2) << beta(m1) 且 m2 显著为负，说明中高SOC区高倍率充电更伤电池")

# ----- 6. 批次内一致性 -----
print("\n" + "=" * 60); print("五、分批次回归(稳健性，批次内用 C1/Q1/C2)"); print("=" * 60)
within_names = ['C1_C', 'Q1_pct', 'C2_C']
for b in ['data_1', 'data_2']:
    sub = std[std['batch'] == b]
    if len(sub) < 10:
        continue
    Xb = sub[within_names].values
    yb = sub['loglife'].values
    Xbs = (Xb - Xb.mean(axis=0)) / Xb.std(axis=0)
    Rb = ols(Xbs, yb)
    show(f"批次 {b} (n={len(sub)}, 标准化)", Rb, within_names)
    print(" 批次内 C2 标准化系数:", f"{Rb['coef'][3]:+.3f}")

# ----- 7. 交互项 -----
print("\n" + "=" * 60); print("六、交互项检验"); print("=" * 60)
std['C1xQ1'] = std['C1_C'] * std['Q1_pct']
X3 = std[['C1_C', 'Q1_pct', 'C2_C', 'T', 'C1xQ1']].values
R3 = ols(X3, std['loglife'].values)
show("加 C1×Q1 交互", R3, ['C1', 'Q1', 'C2', 'T', 'C1×Q1'])

std['C2xQ1'] = std['C2_C'] * std['Q1_pct']
X4 = std[['C1_C', 'Q1_pct', 'C2_C', 'T', 'C1xQ1', 'C2xQ1']].values
R4 = ols(X4, std['loglife'].values)
show("加 C1×Q1 与 C2×Q1 交互", R4, ['C1', 'Q1', 'C2', 'T', 'C1×Q1', 'C2×Q1'])

# ----- 7.5 剂量+批次联合模型 -----
print("\n" + "=" * 60); print("六b、剂量+批次联合模型（log10(寿命) ~ m1 + m2 + batch2)");
    print("=" * 60)
Xdm = std[['m1', 'm2', 'batch2']].values

```

```

Xdms = (Xdm - Xdm.mean(axis=0)) / Xdm.std(axis=0)
Rdm = ols(Xdms, std['loglife'].values)
show("剂量+批次联合模型(标准化)", Rdm, ['m1(低SOC剂量)', 'm2(中高SOC剂量)', 'batch2'])
print(f" m2 与 m1 标准化系数比值: {Rdm['coef'][2]/Rdm['coef'][1]:.2f}")

# ----- 8. 输出参数供论文引用 -----
res = dict(
    beta=std_beta, r2_full=R['r2'], imp=imp,
    m2_beta=R2ms['coef'][2], m1_beta=R2ms['coef'][1],
    m2_beta_joint=Rdm['coef'][2], m1_beta_joint=Rdm['coef'][1],
    r2_m2=R2ms['r2'], m2_pv=R2ms['pv'][2],
    R_full=dict(R), Rr=dict(Rr), Rdm=dict(Rdm),
    within1=dict(ols((std[std['batch']=='data_1'][['C1_C', 'Q1_pct', 'C2_C']].values -
        std[std['batch']=='data_1'][['C1_C', 'Q1_pct', 'C2_C']].values.mean(0)) /
        std[std['batch']=='data_1'][['C1_C', 'Q1_pct', 'C2_C']].values.std(0),
        std[std['batch']=='data_1']['loglife'].values)),
    batch_eff=R['coef'][5], batch_eff_pv=R['pv'][5],
)
np.save(os.path.join(BASE, 'data_processed', 'q2_results.npy'), res, allow_pickle=True)

# ===== 图4: 各变量偏回归图 (控制其他参数与批次后) =====
fig, axes = plt.subplots(1, 4, figsize=(17, 4))
for ax, v in zip(axes, Xr_names):
    i = Xr_names.index(v)
    # 控制其他策略参数 + 批次后的偏回归
    others = [j for j in range(4) if j != i]
    other_cols = [Xr_names[j] for j in others] + ['batch2']
    Xo = np.column_stack([np.ones(len(y)), std[other_cols].values])
    ry = y - Xo @ np.linalg.lstsq(Xo, y, rcond=None)[0]
    rx = Xr[:, i] - Xo @ np.linalg.lstsq(Xo, Xr[:, i], rcond=None)[0]
    ax.scatter(rx, ry, s=30, alpha=.6, c='steelblue')
    z = np.polyfit(rx, ry, 1)
    xx = np.linspace(rx.min(), rx.max(), 50)
    ax.plot(xx, np.polyval(z, xx), 'r--', lw=1.2)
    ax.set_xlabel(f'{v} (残差化)', fontsize=10)
    ax.set_ylabel('log10(寿命) 残差', fontsize=10)
    # 偏相关基于4参数模型
    pr = pcorr(Xr, y, i)
    ax.set_title(f'{v} 偏相关={pr:.2f}', fontsize=10)
fig.suptitle('偏回归图: 控制其他变量与批次后, 各参数与寿命的关系 (standard, n=94)', fontsize=11)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig4_偏回归图.png'), dpi=120); plt.close()

# ===== 图5: 预测寿命热力图 =====
# 用 C1,Q1,C2 全模型在网格上预测 loglife (充电时间用均值)
Tmean = std['T'].mean()
grid_c1 = np.linspace(1, 8, 60)
grid_q1 = np.linspace(5, 80, 60)

```

```

# 固定 C2 = 3.6 与 6.0 两张子图，展示 Q1×C1 面上的寿命
b = Rr['coef']
C1s, Q1s, C2s, Ts = X.mean(axis=0), X.std(axis=0), X.std(axis=0), X.std(axis=0)
def pred(c1, q1, c2):
    return 10 ** (b[0] + b[1]*c1 + b[2]*q1 + b[3]*c2 + b[4]*Tmean)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
for ax, c2 in zip(axes, [3.6, 6.0]):
    Z = np.array([[pred(c1, q1, c2) for q1 in grid_q1] for c1 in grid_c1])
    im = ax.contourf(grid_c1, grid_q1, Z.T, levels=20, cmap='viridis_r')
    ax.set_xlabel('C1 (C)'); ax.set_ylabel('Q1 (%)'); ax.set_title(f'C2 = {c2}C 预测寿命')
    cb = fig.colorbar(im, ax=ax)
    cb.set_label('预测循环寿命')
    # 标记实验覆盖点
    for _, row in std[std['C2_C'] == c2].iterrows():
        ax.plot(row['C1_C'], row['Q1_pct'], 'r.', markersize=5)
fig.suptitle('策略参数空间上的寿命预测（模型：log10(寿命) ~ C1+Q1+C2+T)', fontsize=11)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig5_寿命热力图.png'), dpi=120); plt.close()

print(f"\n图表：fig4_偏回归图.png，fig5_寿命热力图.png")

```

2.5 analysis3.py ——问题三：寿命预测模型

```

# -*- coding: utf-8 -*-
"""B题 问题三：基于早期循环数据的寿命预测模型
特征：早期SOH斜率/波动/潜伏时间/充电时间变化 + 策略参数
模型：Ridge / RandomForest / GradientBoosting, 5折重复交叉验证
评估：不同早期窗口长度 k 的预测误差
"""
import pandas as pd, numpy as np, os, sys, io, warnings
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
warnings.filterwarnings('ignore')
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.model_selection import RepeatedKFold
from sklearn.linear_model import Ridge, LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.preprocessing import StandardScaler

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"

```

```

FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

cy = pd.read_csv(os.path.join(BASE, 'data_processed', '每循环明细表.csv'))
bt = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
bt['cycle_life'] = pd.to_numeric(bt['cycle_life'], errors='coerce')
bt = bt.dropna(subset=['cycle_life']).reset_index(drop=True)
# 充电时间异常清洗 (剔除 >40 min 的脏点)
cy = cy[cy['chargetime_h'] < 40].copy()

# ===== 特征提取 =====
WINDOWS = [5, 10, 20, 50, 100]
THRESH = [99.9, 99.8, 99.7, 99.5, 99.3, 99.0, 98.5, 98.0, 97.0, 96.0]

def extract_features(batt, k):
    d = cy[cy['battery'] == batt].sort_values('cycle').head(k)
    if len(d) < max(3, min(k, 5)):
        return None
    soh = d['SOH_pct'].values.astype(float)
    qd = d['Qd_Ah'].values.astype(float)
    ct = d['chargetime_h'].values.astype(float)
    x = np.arange(1, len(soh) + 1)
    # 线性拟合
    slope, intercept, r, p, se = stats.linregress(x, soh)
    resid = soh - (intercept + slope * x)
    cslope, cint, *_ = stats.linregress(x, ct)
    ctdiff = np.diff(ct)
    feats = {
        'soh_k': soh[-1], 'loss_k': 100 - soh[-1],
        'slope': slope, 'slope_r2': r ** 2,
        'soh_resid_std': np.std(resid), 'soh_firstdiff_std': np.std(np.diff(soh)),
        'qd_std': np.std(qd), 'qd_firstdiff_std': np.std(np.diff(qd)),
        'qd_slope': stats.linregress(x, qd).slope,
        'ct_first': ct[0], 'ct_k': ct[-1], 'ct_slope': cslope,
        'ct_diff_std': np.std(ctdiff), 'ct_drift': ct[-1] - ct[0],
    }
    # 潜伏时间: SOH 首次低于阈值的循环数 (窗口内未达到则记 k+0.5)
    for t in THRESH:
        below = d[d['SOH_pct'] < t]
        feats[f'lat_{t}'] = below['cycle'].min() if len(below) else (k + 0.5)
    return feats

# 提取所有电池特征
recs = []
for batt in bt['battery']:
    for k in WINDOWS:
        f = extract_features(batt, k)

```

```

        if f is None:
            continue
        f['battery'] = batt
        f['window'] = k
        recs.append(f)
F = pd.DataFrame(recs)
# 合并目标与策略参数
F = F.merge(bt[['battery', 'cycle_life', 'C1_C', 'Q1_pct', 'C2_C', 'avg_chargetime_min',
               'batch']],
            on='battery', how='left')
print(f"特征提取完成: {len(F)} 条 (电池x窗口)")

# ===== 建模评估 =====
DEG_FEATS = ['soh_k', 'loss_k', 'slope', 'slope_r2', 'soh_resid_std', 'soh_firstdiff_std',
             'qd_std', 'qd_firstdiff_std', 'qd_slope',
             'ct_first', 'ct_k', 'ct_slope', 'ct_diff_std', 'ct_drift'] + [f'lat_{t}' for t in
                                THRESH]
STRAT_FEATS = ['C1_C', 'Q1_pct', 'C2_C', 'avg_chargetime_min']
ALL_FEATS = DEG_FEATS + STRAT_FEATS

def make_model(name):
    if name == 'Ridge':
        return Ridge(alpha=1.0)
    if name == 'RandomForest':
        return RandomForestRegressor(n_estimators=300, min_samples_leaf=2, random_state=42,
                                     n_jobs=-1)
    return GradientBoostingRegressor(n_estimators=250, max_depth=2, learning_rate=0.05,
                                     random_state=42)

def cv_evaluate(feats_cols, label, model_name, k, seed=42):
    sub = F[(F['window'] == k) & (F[feats_cols].notna().all(axis=1))].reset_index(drop=True)
    X = sub[feats_cols].values
    y1 = np.log10(sub['cycle_life'].values)
    rkf = RepeatedKFold(n_splits=5, n_repeats=8, random_state=seed)
    preds, trues = [], []
    for tr, te in rkf.split(X):
        m = make_model(model_name)
        sc = StandardScaler().fit(X[tr])
        Xtr, Xte = sc.transform(X[tr]), sc.transform(X[te])
        m.fit(Xtr, y1[tr])
        preds.append(m.predict(Xte))
        trues.append(y1[te])
    preds = np.concatenate(preds); trues = np.concatenate(trues)
    life_p = 10 ** preds; life_t = 10 ** trues
    mape = np.mean(np.abs(life_p - life_t) / life_t) * 100
    r2 = r2_score(trues, preds)
    return dict(mape=mape, r2=r2)

```

```

print("\n" + "=" * 60)
print("一、不同早期窗口长度下的预测误差 (5折*8次重复CV)")
print("=" * 60)
print(f"{'窗口k':>6}{'模型':<14}{'MAPE(%)':>10}{'R²(log10)':>12}")
summary = {}
for k in WINDOWS:
    best = None
    for mn in ['Ridge', 'RandomForest', 'GradientBoosting']:
        r = cv_evaluate(ALL_FEATS, 'all', mn, k)
        summary[(k, mn)] = r
        print(f"{'k':>6}{'mn':<14}{'r['mape']':>10.1f}{'r['r2']':>12.3f}")
        if best is None or r['mape'] < best[1]:
            best = (mn, r['mape'], r['r2'])
    print(f"    最优: {best[0]} MAPE={best[1]:.1f}% R²={best[2]:.3f}")
    print("-" * 50)

# ===== 特征组合消融 (k=100) =====
print("\n" + "=" * 60)
print("二、特征组合消融 (k=100, GradientBoosting)")
print("=" * 60)
for label, feats in [('仅早期衰减特征', DEG_FEATS),
                     ('仅策略参数', STRAT_FEATS),
                     ('早期特征+策略参数', ALL_FEATS)]:
    r = cv_evaluate(feats, label, 'GradientBoosting', 100)
    print(f" {label:<16}: MAPE={r['mape']:.1f}% R²={r['r2']:.3f}")

# ===== 最佳模型：拟合与特征重要性 (k=100) =====
sub = F[(F['window'] == 100) & (F[ALL_FEATS].notna().all(axis=1))].reset_index(drop=True)
X = sub[ALL_FEATS].values
yl = np.log10(sub['cycle_life'].values)
g = GradientBoostingRegressor(n_estimators=250, max_depth=2, learning_rate=0.05,
                              random_state=42)
g.fit(X, yl)
pred = g.predict(X)
imp = pd.Series(g.feature_importances_, index=ALL_FEATS).sort_values(ascending=False)
print("\n" + "=" * 60)
print("三、特征重要性 (GradientBoosting, k=100, 全特征拟合)")
print("=" * 60)
print(imp.head(15).to_string())
print(f"\n训练R²={r2_score(yl, pred):.3f}")

# 保存预测表 (对每个电池在k=100的留一/全拟合预测)
sub['pred_life'] = 10 ** pred
sub.to_csv(os.path.join(BASE, 'data_processed', 'q3_predictions.csv'), index=False,
           encoding='utf-8-sig')
# 保存结果

```

```

res = dict(summary={f'{k}_{m}': v for (k, m), v in summary.items()},
            imp=imp.to_dict(), feats=ALL_FEATS, windows=WINDOWS)
np.save(os.path.join(BASE, 'data_processed', 'q3_results.npy'), res, allow_pickle=True)

# ===== 图6: 预测 vs 实际 (k=100, GBM, 交叉验证预测) =====
# 用 RepeatedKfold 收集 OOF 预测
rkf = RepeatedKfold(n_splits=5, n_repeats=10, random_state=42)
oof = np.zeros(len(sub))
for tr, te in rkf.split(X):
    m = GradientBoostingRegressor(n_estimators=250, max_depth=2, learning_rate=0.05,
                                   random_state=42)
    m.fit(X[tr], y1[tr])
    oof[te] = m.predict(X[te])
life_t = 10 ** y1; life_p = 10 ** oof
fig, ax = plt.subplots(figsize=(7, 6.5))
ax.scatter(life_t, life_p, c=sub['batch'].map({'data_1': 'steelblue', 'data_2': 'darkorange',
        'data_3': 'green'}), s=35, alpha=.75)
lim = [100, 2200]
ax.plot(lim, lim, 'k--', lw=1)
ax.plot(lim, [x * 0.9 for x in lim], 'k:', lw=.8); ax.plot(lim, [x * 1.1 for x in lim], 'k:',
        lw=.8)
ax.set_xlim(lim); ax.set_ylim(lim); ax.set_xlabel('实际循环寿命'); ax.set_ylabel('预测循环寿命')
mape_cv = np.mean(np.abs(life_p - life_t) / life_t) * 100
r2_cv = r2_score(y1, oof)
ax.set_title(f'早期100循环预测寿命 (GBM, 交叉验证)\nMAPE={mape_cv:.1f}%, R²={r2_cv:.3f},
        n={len(sub)}')
from matplotlib.lines import Line2D
handles = [Line2D([0], [0], marker='o', color='none', markerfacecolor=c, markersize=8, label=b)
        for b, c in [('data_1', 'steelblue'), ('data_2', 'darkorange'), ('data_3', 'green')]]
ax.legend(handles=handles, loc='upper left', fontsize=9)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig6_寿命预测对比.png'), dpi=120);
plt.close()

# ===== 图7: MAPE vs 早期窗口长度 =====
fig, ax = plt.subplots(figsize=(7.5, 5))
for mn, c, mk in [('Ridge', 'steelblue', 'o'), ('RandomForest', 'darkorange', 's'),
        ('GradientBoosting', 'green', '^')]:
    vals = [summary[k, mn]['mape'] for k in WINDOWS]
    ax.plot(WINDOWS, vals, marker=mk, color=c, lw=1.8, label=mn)
ax.set_xlabel('早期循环窗口长度 k (个循环)'); ax.set_ylabel('寿命预测 MAPE (%)')
ax.set_xticks(WINDOWS); ax.grid(alpha=.3); ax.legend()
ax.set_title('预测误差随早期数据长度变化 (5折×8重复交叉验证)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig7-MAPE-vs窗口.png'), dpi=120); plt.close()

# ===== 图8: 特征重要性 =====
fig, ax = plt.subplots(figsize=(7.5, 6))
imp_top = imp.head(12)[::-1]

```



```

ax.barh(imp_top.index, imp_top.values, color='steelblue')
ax.set_xlabel('特征重要性'); ax.set_title('寿命预测特征重要性 (k=100, GradientBoosting)')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig8_特征重要性.png'), dpi=120); plt.close()

print(f"\n图表: fig6_寿命预测对比.png, fig7_MAPE_vs窗口.png, fig8_特征重要性.png")

# ===== 四、SOH 未来轨迹预测 (里程碑循环) =====
print("\n" + "=" * 60)
print("四、早期数据预测未来 SOH (k=20 特征, 里程碑循环 t=200/400/600/800/1000)")
print("=" * 60)
MILESTONES = [200, 400, 600, 800, 1000]
# 每个电池的 SOH@t (取最接近 t 的循环)
soh_at = {}
for batt in bt['battery']:
    d = cy[cy['battery'] == batt].sort_values('cycle')
    vals = {}
    for t in MILESTONES:
        near = d.iloc[(d['cycle'] - t).abs().argmin()]
        if abs(near['cycle'] - t) <= 5:
            vals[t] = near['SOH_pct']
        else:
            vals[t] = np.nan
    soh_at[batt] = vals
Fs = F[F['window'] == 20].reset_index(drop=True)
traj_res = {}
for t in MILESTONES:
    Fs[f'soh_{t}'] = Fs['battery'].map(lambda b: soh_at[b][t])
    tr = Fs.dropna(subset=[f'soh_{t}'] + ALL_FEATS).reset_index(drop=True)
    if len(tr) < 15:
        continue
    X = tr[ALL_FEATS].values
    y = tr[f'soh_{t}'].values
    rkf = RepeatedKFold(n_splits=5, n_repeats=8, random_state=7)
    oof = np.zeros(len(tr))
    for tt, te in rkf.split(X):
        m = GradientBoostingRegressor(n_estimators=200, max_depth=2, learning_rate=0.05,
                                      random_state=42)
        m.fit(X[tt], y[tt]); oof[te] = m.predict(X[te])
    mae = mean_absolute_error(y, oof)
    r2s = r2_score(y, oof)
    traj_res[t] = dict(mae=mae, r2=r2s, n=len(tr))
    print(f" 里程碑 t={t}: R²={r2s:.3f} MAE={mae:.2f} pct (n={len(tr)})")

# ===== 图9: 预测SOH vs 实际SOH 轨迹 =====
# 挑选 3 个代表性电池 (长/中/短寿命), 展示早期20循环数据 + 里程碑预测 + 实际轨迹
sample = ['data_1_cell100', 'data_2_cell109', 'data_2_cell101'] # 长/中/短
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5), sharey=True)

```

```

for ax, batt in zip(axes, sample):
    d = cy[cy['battery'] == batt].sort_values('cycle')
    cl = bt[bt['battery'] == batt]['cycle_life'].values[0]
    ax.plot(d['cycle'], d['SOH_pct'], color='lightgray', lw=1.2, label='实际SOH轨迹')
    d20 = cy[cy['battery'] == batt].sort_values('cycle').head(20)
    ax.plot(d20['cycle'], d20['SOH_pct'], 'b-', lw=2, label='前20循环(训练用)')
    # 里程碑真实 SOH (预测目标)
    for t in MILESTONES:
        v = soh_at[batt][t]
        if np.isfinite(v):
            ax.plot(t, v, 'ro', ms=6)
    ax.axhline(80, color='red', ls='--', alpha=.5)
    ax.set_title(f'{batt}\ncycle_life={int(cl)}')
    ax.set_xlabel('循环数')
axes[0].set_ylabel('SOH (%)')
handles, labels = axes[0].get_legend_handles_labels()
axes[2].legend(handles, labels, fontsize=8, loc='lower left')
plt.suptitle('实际 SOH 轨迹与前 20 循环数据 (用于早期预测)', fontsize=11)
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig9_轨迹预测演示.png'), dpi=120);
plt.close()

np.save(os.path.join(BASE, 'data_processed', 'q3_traj.npy'), traj_res, allow_pickle=True)
print("\n图9: fig9_轨迹预测演示.png")

```

2.6 analysis4.py ——问题四：策略优化

```

# -*- coding: utf-8 -*-
"""B题 问题四：兼顾充电时间与寿命衰减的快充策略优化
1) 充电时间模型 (物理理想模型 + 经验校准)
2) 寿命模型: SOC剂量模型  $\log_{10}(\text{寿命}) \sim m_1 + m_2$ ,  $m_1=C1*Q1/100$ ,  $m_2=C2*(80-Q1)/100$ 
3) 在观测参数水平构成的"合理邻域"内网格搜索, 提取 Pareto 前沿
4) 推荐策略 + 与典型长/短寿命策略对比
"""
import pandas as pd, numpy as np, os, sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

BASE = r"C:\Users\one\Desktop\2026年校赛题目\B"
FIG = os.path.join(BASE, 'data_processed', 'figs')
os.makedirs(FIG, exist_ok=True)

```

```

bt = pd.read_csv(os.path.join(BASE, 'data_processed', 'battery_table.csv'))
bt['cycle_life'] = pd.to_numeric(bt['cycle_life'], errors='coerce')
std = bt[bt['protocol'] == 'standard'].dropna(subset=['cycle_life']).copy()
std['loglife'] = np.log10(std['cycle_life'])
std['m1'] = std['C1_C'] * std['Q1_pct'] / 100
std['m2'] = std['C2_C'] * (80 - std['Q1_pct']) / 100
print(f"standard 电池: {len(std)}")

# ----- 1. 充电时间校准模型 -----
A = np.column_stack([std['Q1_pct']/100/std['C1_C'], (80-std['Q1_pct'])/100/std['C2_C'],
                    np.ones(len(std))])
yT = std['avg_chargetime_min'].values
w = np.linalg.lstsq(A, yT, rcond=None)[0]
Tpred = A @ w
T_r2 = 1 - np.sum((yT - Tpred)**2) / np.sum((yT - yT.mean())**2)
print(f"\n[1] 充电时间校准模型: T = {w[0]:.2f}*(Q1/100)/C1 + {w[1]:.2f}*(80-Q1)/100/C2 +
      {w[2]:.2f}")
print(f"  校准 R²={T_r2:.3f}, MAE={np.mean(np.abs(yT-Tpred)):.2f} min (理想模型系数为60)")

def T80(C1, Q1, C2):
    return w[0]*(Q1/100)/C1 + w[1]*((80-Q1)/100)/C2 + w[2]

# ----- 2. 剂量寿命模型 -----
print("\n[2] 剂量寿命模型 log10(寿命) ~ m1 + m2")
life_res = {}
for tag, s in [('全部(standard)', std), ('批次data_1', std[std['batch']=='data_1']),
              ('批次data_2', std[std['batch']=='data_2'])]:
    A2 = np.column_stack([np.ones(len(s)), s['m1'], s['m2']])
    b, *_ = np.linalg.lstsq(A2, s['loglife'], rcond=None)
    p = A2 @ b
    r2 = 1 - np.sum((s['loglife']-p)**2)/np.sum((s['loglife']-s['loglife'].mean())**2)
    life_res[tag] = dict(b=b, r2=r2, n=len(s))
    print(f"  {tag}<14: 截距={b[0]:.3f}, m1={b[1]:.4f}, m2={b[2]:.4f}, R²={r2:.3f},
          n={len(s)}")

# 优化用: 全部 standard 的剂量模型 (一般性设计建议)
b_all = life_res['全部(standard)']['b']
def life_model(C1, Q1, C2):
    m1 = C1*Q1/100; m2 = C2*(80-Q1)/100
    return 10 ** (b_all[0] + b_all[1]*m1 + b_all[2]*m2)

# ----- 3. 网格搜索 (观测水平合理邻域 + 剂量覆盖约束) -----
C1_lv = np.unique(std['C1_C'])
Q1_lv = np.unique(std['Q1_pct'])
C2_lv = np.unique(std['C2_C'])
C1m, Q1m, C2m = np.meshgrid(C1_lv, Q1_lv, C2_lv, indexing='ij')

```

```

m1_all = C1m*Q1m/100
m2_all = C2m*(80-Q1m)/100
T_all = T80(C1m, Q1m, C2m)
L_all = life_model(C1m, Q1m, C2m)

# 剂量覆盖约束: 候选设计的 (m1,m2) 必须落在实测剂量的凸包范围附近, 排除 8C(80%) 等未验证角点
m1_obs = std['m1'].values; m2_obs = std['m2'].values
m1_lo, m1_hi = m1_obs.min(), m1_obs.max()
m2_lo, m2_hi = m2_obs.min(), m2_obs.max()
feasible = (m1_all >= m1_lo) & (m1_all <= m1_hi) & (m2_all >= m2_lo) & (m2_all <= m2_hi)
n_all = C1m.size
n_feas = feasible.sum()
print(f"\n[3] 网格规模: {len(C1_lv)}*{len(Q1_lv)}*{len(C2_lv)} = {n_all} 个候选")
print(f" 剂量约束 (m1 [{m1_lo:.2f},{m1_hi:.2f}], m2 [{m2_lo:.2f},{m2_hi:.2f}]) 后可行: {n_feas} 个")

# Pareto 非支配点 (暴力筛选: 最大化寿命, 最小化充电时间)
Tf = T_all[feasible]; Lf = L_all[feasible]
C1f = C1m[feasible]; Q1f = Q1m[feasible]; C2f = C2m[feasible]
keep = np.ones(len(Tf), bool)
for i in range(len(Tf)):
    if not keep[i]:
        continue
    for j in range(len(Tf)):
        if i != j and Lf[j] >= Lf[i] and Tf[j] <= Tf[i] and (Lf[j] > Lf[i] or Tf[j] < Tf[i]):
            keep[i] = False
            break
Tf, Lf, C1f, Q1f, C2f = Tf[keep], Lf[keep], C1f[keep], Q1f[keep], C2f[keep]
P = pd.DataFrame(dict(C1=C1f, Q1=Q1f, C2=C2f, T=Tf,
                      life=Lf)).drop_duplicates().sort_values('T')
print(f" Pareto 前沿点数: {len(P)}")

# 膝点: 归一化(寿命取对数)空间上离端点连线最远的点
def knee_point(P):
    TN = (P['T'] - P['T'].min())/(P['T'].max() - P['T'].min())
    L = np.log10(P['life'])
    LN = (L - L.min())/(L.max() - L.min())
    A = np.column_stack([TN.values, LN.values])
    p1, p2 = A[0], A[-1]
    v = p2 - p1; vn = np.linalg.norm(v)
    # 点到端点连线距离 (2D 叉积)
    d = np.abs(v[0]*(A[:,1]-p1[1]) - v[1]*(A[:,0]-p1[0])) / vn
    return int(np.argmax(d)), d
kidx, darr = knee_point(P)
rec = P.iloc[kidx]
print(f" 膝点(推荐): C1={rec['C1']:.1f}C, Q1={rec['Q1']:.0f}%, C2={rec['C2']:.1f}C")
print(f" T80={rec['T']:.2f} min, 预测寿命={rec['life']:.0f}")

```

```

# 备选：充电时间预算约束下寿命最大（更快的方案）
_fastsub = P.loc[P['T'] <= 12.7]
fast = _fastsub.loc[_fastsub['life'].idxmax()]
print(f" 备选(更快, T<=12.7min): C1={fast['C1']:.1f}C, Q1={fast['Q1']:.0f}%,
      C2={fast['C2']:.1f}C, "
      f"T80={fast['T']:.2f} min, 预测寿命={fast['life']:.0f}")
# 若退化为与膝点相同，则固定为数据验证过的 4C(80%)-4C
if abs(fast['T'] - rec['T']) < 0.01:
    fast = dict(C1=4.0, Q1=80, C2=4.0, T=T80(4.0, 80, 4.0), life=life_model(4.0, 80, 4.0))
    print(f" 备选(固定为 4C(80%)-4C): T80={fast['T']:.2f} min, 预测寿命={fast['life']:.0f}")

# ----- 4. 对比 -----
fmt = lambda v: f'{v:g}' # 4.0 -> '4', 3.6 -> '3.6'
print("\n[4] 推荐策略与典型策略对比（预测 + 实测）")
print(f"{'策略':<26}{'C1(C)':>6}{'Q1(%)':>6}{'C2(C)':>6}{'T80(min)':>9}{'预测寿命':>9}{'实测寿命':>9}")
rows = [
    ('推荐(膝点)', rec['C1'], rec['Q1'], rec['C2'], None),
    ('备选(更快)', fast['C1'], fast['Q1'], fast['C2'], None),
    ('长寿命 3.6C(80%)-3.6C', 3.6, 80, 3.6, None),
    ('长寿命 4C(80%)-4C', 4.0, 80, 4.0, None),
    ('长寿命 8C(15%)-3.6C', 8.0, 15, 3.6, None),
    ('6C(30%)-3.6C', 6.0, 30, 3.6, None),
    ('短寿命 1C(4%)-6C', 1.0, 4, 6.0, None),
    ('短寿命 2C(10%)-6C', 2.0, 10, 6.0, None),
]
for name, c1, q1, c2, _ in rows:
    T = T80(c1, q1, c2); L = life_model(c1, q1, c2)
    pol = f'{fmt(c1)}C({int(q1)}%)-{fmt(c2)}C'
    obs = std[std['policy'] == pol]
    obs_life = f"{obs['cycle_life'].mean():.0f}" if len(obs) else '—'
    print(f"{name:<26}{c1:>6.1f}{q1:>6.0f}{c2:>6.1f}{T:>9.1f}{L:>9.0f}{obs_life:>9}")

# ----- 5. 与数据集中最优策略(按实测寿命/充电时间)对比 -----
print("\n[5] 数据集中 standard 各策略的平均实测寿命与平均充电时间")
pol_agg = std.groupby('policy').agg(n=('cycle_life', 'size'), life=('cycle_life', 'mean'),
                                    T=('avg_chargetime_min', 'mean')).reset_index()
best_life = pol_agg.sort_values('life', ascending=False).head(3)
print("实测寿命最高的3个策略:")
print(best_life.to_string(index=False))

# 保存
res = dict(w=w, T_r2=T_r2, b_all=b_all, life_res={k: {kk: (vv.tolist() if isinstance(vv,
np.ndarray) else vv)
                                                    for kk, vv in v.items()}} for k, v in
life_res.items()),
front=P[['C1', 'Q1', 'C2', 'T', 'life']].to_dict('records'), rec=rec.to_dict(),

```

```

        fast=fast.to_dict())
np.save(os.path.join(BASE, 'data_processed', 'q4_results.npy'), res, allow_pickle=True)

# ===== 图10: Pareto 前沿 =====
fig, ax = plt.subplots(figsize=(8, 6.5))
ax.scatter(T_all.ravel(), L_all.ravel(), s=4, alpha=.25, c='lightsteelblue',
           label='可行设计(观测水平邻域)')
ax.plot(P['T'], P['life'], 'o-', color='crimson', ms=6, lw=1.6, label='Pareto 前沿')
ax.plot(rec['T'], rec['life'], 'r*', ms=20, label=f'推荐策略(膝点):
        {rec["C1"]:.1f}C({rec["Q1"]:.0f}%) - {rec["C2"]:.1f}C')
ax.set_xlabel('充电时间 T80 (min)'); ax.set_ylabel('预测循环寿命')
ax.legend(fontsize=9, loc='upper right')
ax.set_title('快充策略 Pareto 前沿 (充电时间 vs 预测寿命)\n寿命模型: log10(寿命) ~ 低SOC剂量 +
        中高SOC剂量')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig10_Pareto前沿.png'), dpi=120); plt.close()

# ===== 图11: 推荐充电策略电流剖面 =====
fig, ax = plt.subplots(figsize=(8, 4.6))
c1, q1, c2 = rec['C1'], rec['Q1'], rec['C2']
soc = np.array([0, q1, 80, 100]); I = np.array([c1, c2, 1.0, 0.0])
ax.step(soc, I, where='post', lw=2.5, color='steelblue')
ax.axvline(q1, color='gray', ls=':', lw=1); ax.text(q1, c1+0.25, f'Q1={q1:.0f}%', ha='center',
           fontsize=9)
ax.text(q1/2, c1+0.2, f'{c1:.1f}C', ha='center', fontsize=10)
ax.text((q1+80)/2, c2+0.2, f'{c2:.1f}C', ha='center', fontsize=10)
ax.text(90, 1.1, '1C CC-CV', ha='center', fontsize=9)
ax.set_xlabel('SOC (%)'); ax.set_ylabel('充电倍率 (C)'); ax.set_xlim(0,100); ax.set_ylim(0,
           c1+1)
ax.set_title(f'推荐快充策略: {c1:.1f}C({q1:.0f}%) - {c2:.1f}C\nT80 {T80(c1,q1,c2):.1f} min,
           预测寿命 {life_model(c1,q1,c2):.0f}')
plt.tight_layout(); plt.savefig(os.path.join(FIG, 'fig11_推荐策略剖面.png'), dpi=120);
plt.close()

print(f"\n图表: fig10_Pareto前沿.png, fig11_推荐策略剖面.png")

```